# Rapport de Soutenance




## Par Rabbit Hole

Le Team Lead
*Coordinateur de l'équipe, Graphiste, Développeur*

Le Deputy Lead
*Coordinateur Adjoint, Développeur*

Noah FAUVEL
*Développeur*

Le Développeur A
*Développeur*

Soutenance finale – mercredi 26 mai 2026

# Sommaire

**1 Introduction....4**

**2 Avancements Soutenance 1....5**

2.1 Organisation et mise en route....5

2.1.1 Mise en place de réunions....5

*2.1.1.1 Bilan du Team Lead....5*

2.1.2 Choix des plateformes en ligne....6

2.1.3 Choix du moteur....7

2.2 Assets....7

2.2.1 Bilan du Team Lead....7

2.2.2 Bilan du Deputy Lead....8

2.3 Mouvement....8

2.3.1 Bilan du Développeur A....8

2.3.2 Bilan Noah....9

2.3.3 Bilan du Deputy Lead....9

2.4 Multijoueur....10

2.4.1 Bilan du Deputy Lead....10

2.5 IA des poissons....11

2.5.1 Bilan du Team Lead....11

2.5.2 Bilan Noah....11

2.6 Map....12

2.6.1 Bilan du Team Lead....12

2.7 Site Web....12

2.7.1 Bilan du Team Lead....12

2.7.2 Bilan du Développeur B....12

**3 Avancements Soutenance 2....14**

3.1 Menu....14

3.1.1 Bilan du Deputy Lead....14

3.1.2 Bilan du Développeur A....14

3.2 Physique....16

3.2.1 Bilan Noah....16

3.3 Network....17

3.3.1 Bilan du Deputy Lead....17

3.4 IA des poissons....18

3.4.1 Bilan du Team Lead....18

3.4.2 Bilan Noah....19

3.5 Pêche....19

3.5.1 Bilan du Développeur A....19

3.6 Assets....23

3.6.1 Bilan du Team Lead....23

3.6.2 Bilan du Deputy Lead....23

3.7 Site Web....24

3.7.1 Bilan du Développeur B....24

3.8 Tchat..... 25
3.8.1 Bilan du Deputy Lead.....25
**4 Futures tâches.....27**
4.1 Menu..... 27
4.1.1 Bilan du Deputy Lead.....27
4.2 Pêche.....27
4.2.1 Bilan du Développeur A.....27
4.3 Inventaire..... 27
4.3.1 Bilan du Développeur A.....27
4.4 Boutiques.....27
4.4.1 Bilan Noah.....27
4.5 Organisation..... 28
**5 Avancements Soutenance 3.....29**
5.1 Bilan UI..... 29
5.1.1 Bilan du Deputy Lead.....29
5.1.2 Bilan du Développeur A.....30
5.2 Pêche.....31
5.2.1 Bilan du Développeur A.....31
5.3 Shop..... 32
5.3.1 Bilan du Développeur A.....32
5.3.2 Bilan du Deputy Lead.....33
5.4 Sauvegarde.....34
5.4.1 Bilan du Deputy Lead.....34
5.5 Animations.....34
5.5.1 Bilan du Développeur A.....34
5.6 Assets.....35
5.6.1 Bilan du Développeur A.....35
5.6.2 Bilan du Team Lead.....36
*5.6.2.1 Assets 3D.....36*
*5.6.2.2 Assets 2D.....37*
5.6.3 Bilan du Deputy Lead.....38
5.7 Musiques..... 38
5.7.1 Bilan du Team Lead.....38
5.8 Rendus et packaging..... 39
5.8.1 Bilan du Team Lead.....39
5.9 Physique..... 40
5.9.1 Bilan Noah.....40
5.10 Organisation..... 40
**6 Conclusion.....42**

# 1 Introduction

Dans notre jeu, le joueur incarne un lapin pêcheur qui utilise un fusil à pompe pour attraper des poissons.

La principale mécanique du jeu est donc la pêche ; elle se déroule sous forme de combat. Lorsque nous sommes assez proches d'un point de pêche, nous pouvons interagir avec celui-ci et entrer en combat.

Lors de ce combat, il est impossible de discerner les poissons : nous pouvons seulement voir leur forme et donc leurs espèces.

Pour pêcher un poisson, il faut suivre le poisson jusqu'à charger une jauge au maximum pour pouvoir tirer et capturer le poisson.

Une fois capturé, ce poisson va s'ajouter au Fishodex, un livre qui nous permet de catégoriser tous les poissons que nous avons pêchés.

Pour compléter ce Fishodex, nous pourrons nous aider de différentes améliorations disponibles dans le jeu comme améliorer notre arme ou encore notre chance de croiser des poissons plus rares.

Il y aura également un magasin disponible qui nous permettra de racheter des munitions ou même d'échanger des poissons voulus avec d'autres joueurs.

Enfin, notre jeu ne sera pas seulement pêche et combat : il y aura une carte sur laquelle nous pouvons nous balader dans un décor calme et apaisant et un chat écrit qui nous permettra d'interagir avec d'autres joueurs.

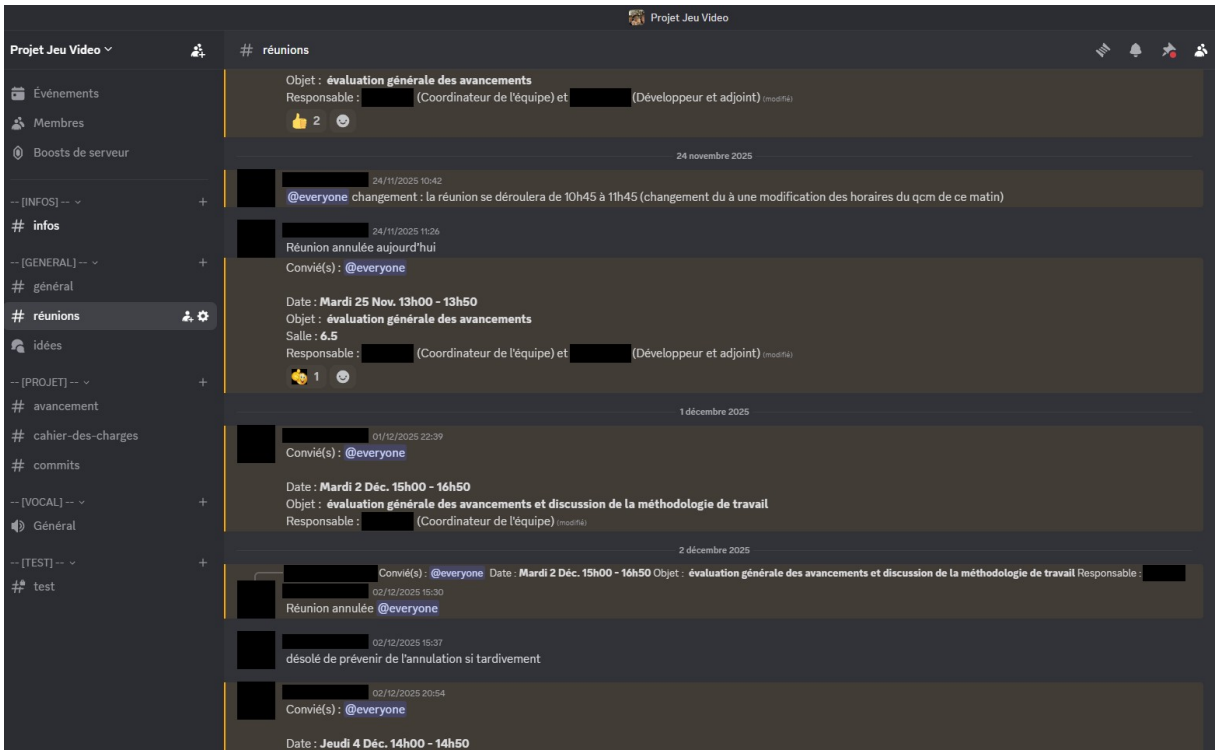
# 2 Avancements Soutenance 1

## 2.1 Organisation et mise en route

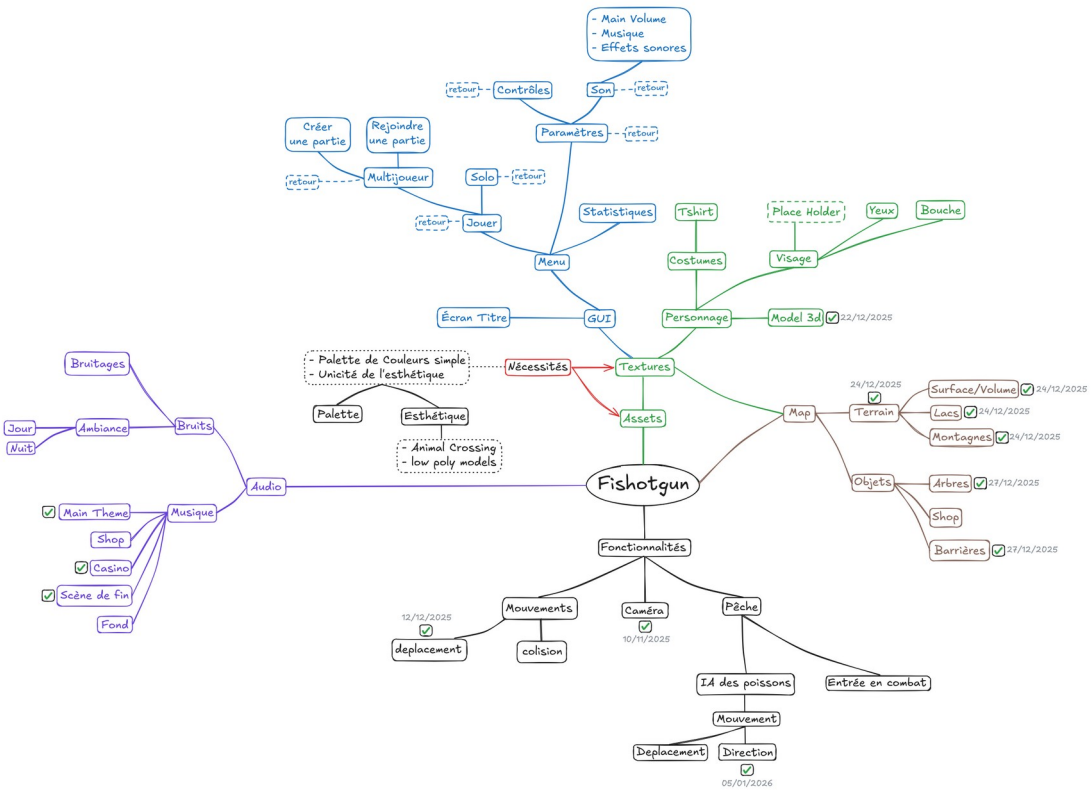
### 2.1.1 Mise en place de réunions

#### *2.1.1.1 Bilan du Team Lead*

Pour ma part, le début du projet consistait principalement à préparer des réunions (voir image ci-dessous) pour organiser la répartition des tâches par personne et dans le temps. Comme expliqué dans la partie 2.2, nous avons utilisé différentes plateformes de communication pour organiser cette planification.




Après la répartition des rôles, j'ai commencé à planifier les premières réunions ainsi que les thèmes qu'elles abordent. Nous avons réfléchi à une méthodologie de travail adaptée à tous. J'ai créé une carte mentale pour situer visuellement les objectifs principaux, secondaires… ainsi que leur avancement (voir image ci-dessous).




### 2.1.2 Choix des plateformes en ligne

Au début du projet, après la création du groupe et l'attribution de rôles à chaque membre, notre premier objectif a été de mettre en place des moyens de communication et de planification des tâches pour le projet.

Pour la discussion, la présentation d'idées, le partage des avancements de chacun et des différentes présentations de groupe ainsi que pour l'annonce et le suivi des réunions, nous avons privilégié Discord.

Cette plateforme de messagerie nous a permis de catégoriser ces différents types d'échanges en salons de discussion. Elle nous permet également d'être en appel vocal pour travailler à distance. Elle est un point de repère pour toute l'équipe pour parcourir l'historique de ce qui a été fait et pour discuter des prochains objectifs à réaliser.

Pour la partie logiciel (partage du code) et répartition des tâches, nous avons choisi GitHub. Cette plateforme nous permet également d'avoir un diagramme de Gantt (voir annexe), récapitulant les différents avancements dans le temps, ceux qui en sont responsables ainsi que la date des différentes soutenances qui nous servent de repères.

Nous avons également lié notre projet sur GitHub à notre Discord de manière à être notifiés, dans un salon dédié, des derniers commits faits. Ces deux plateformes sont celles que nous utilisons le plus car elles centralisent la communication et la planification du projet.

Nous avons, à la suite de la réunion du 22 décembre concernant l'évaluation générale des avancements, pris la décision de placer sur une carte mentale l'organisation des différents points et objectifs du projet.

Pour cela nous avons choisi Excalidraw (voir image p.4/12), un site web dédié à la création dynamique (modifiable par différents utilisateurs en même temps) de mindmaps à plusieurs.

Le concept de carte mentale nous permet d'avoir un repère visuel ainsi que de séparer les objectifs en sous-objectifs de manière à les répartir plus distinctement dans le temps et à mieux appréhender le travail à faire.

### 2.1.3 Choix du moteur

Nous étions d'abord partis sur Panda3D comme moteur de jeu, mais nous avons par la suite dérivé sur Ursina, qui abstrait Panda3D. Cela permet d'utiliser plus facilement les outils proposés par Panda3D, car Ursina fait la plupart des initialisations nécessaires au bon fonctionnement de Panda3D.

Aussi, Ursina est codé en Python, ce qui permet d'avoir directement accès au code source et de mieux comprendre son fonctionnement.

Malgré ses nombreux avantages, Ursina manque de documentation, ce qui rend son apprentissage complexe. Lors de notre choix de moteur de jeu, ce défaut a été outrepassé par la facilité d'utilisation de l'interface de programmation (API) d'Ursina.

## 2.2 Assets

### 2.2.1 Bilan du Team Lead

Je me suis rapidement attelé à la conception de l'identité visuelle du jeu et de l'entreprise, avec la création d'un logo et d'une palette de couleurs pour chacun. Après de nombreux jets, nous nous sommes finalement mis d'accord sur le logo actuel que le Deputy Lead et moi avons fait ensemble. Pour ces logos, j'ai utilisé Photoshop.

Le Deputy Lead s'est occupé de modéliser le modèle 3D du lapin en se basant en partie sur des croquis que je lui ai fournis.

J'ai ensuite travaillé sur les textures du modèle. J'en ai fait sur Photoshop des visages et des couleurs pour certaines parties du corps (oreilles roses, corps blanc). Problème : le maillage du modèle 3D du Deputy Lead a mal été réalisé et rend compliqué l'aplat de textures sur des zones précises. Il faudra donc refaire le modèle afin d'avoir une UV-map plus propre et facile à utiliser pour les textures.

Pour le fishobus, j'ai fait des rendus de face et de profil (toujours sur Photoshop) afin de pouvoir réaliser son modèle plus facilement sur Blender après. J'ai fait différents croquis pour déterminer comment le colorier et ai demandé l'avis de membres de l'équipe dessus.

Pour ce qui est des musiques, je m'inspire beaucoup des musiques d'Animal Crossing et de Webfishing pour avoir des musiques calmes et relaxantes dans un style bossa nova. La musique du potentiel casino est une exception. Inspirée du jeu Grim Fandango, elle penche plus vers un style de jazz bebop.

Pour la composition de ces dernières, je trouve les mélodies à la guitare avant de les mettre au propre sur FL Studio. J'utilise alors des instruments synthétisés pour contribuer à placer Fishotgun dans le style de certains jeux du début des années 2000. Ceci va de pair avec le rendu « low poly » et pixelisé des modèles 3D et des textures.

### 2.2.2 Bilan du Deputy Lead

Fin octobre, j'ai commencé à apprendre la modélisation 3D à l'aide de Blender, à travers les tutos de « Joey Carlino » et ceux de « Grant Abbitt », qui m'ont permis de réaliser le premier modèle du personnage. Ce modèle a ensuite été revu, car il n'allait pas avec la direction artistique choisie et était beaucoup trop réaliste. Cependant, ce premier travail m'a permis de le refaire beaucoup plus rapidement.

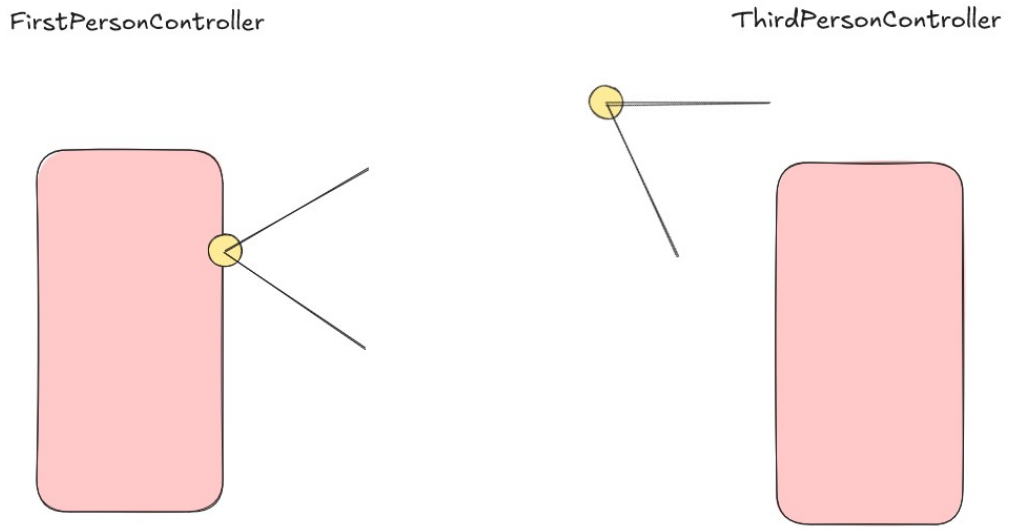
## 2.3 Mouvement

### 2.3.1 Bilan du Développeur A

Pour commencer, j'ai établi la base du mouvement de notre jeu. J'ai utilisé un système préexistant d'Ursina que j'ai manipulé selon mes souhaits pour établir une entité que l'on puisse faire se déplacer et sauter.

Ce système est celui du FirstPersonController, nous avons voulu le réutiliser afin de faciliter la mise en place des mouvements du joueur.

Cependant, bien qu'il soit fonctionnel et pratique, il présente un défaut majeur, étant basé sur la première personne, alors que nous voulions un jeu en troisième personne. Ainsi, j'ai manipulé l'origine et le déplacement de la caméra pour que ça marche comme on le souhaitait, et j'en ai créé une nouvelle classe du nom de ThirdPersonController.




Un problème s'était imposé : celui de la caméra qui avait une collision ; ainsi, le joueur montait indéfiniment vers le haut lorsqu'on pointait la caméra vers le haut (celle-ci étant derrière le joueur).

On a pu le résoudre en supprimant les collisions de cette caméra, mais on a dû implémenter un autre système pour éviter que la caméra rentre dans le sol.

Pour ce qui est du mouvement, celui de base d'Ursina est fonctionnel, mais ça ne marchait pas comme on le voulait. J'ai donc modifié les propriétés du joueur afin que sa gravité, sa vitesse et la fluidité dans le mouvement soient adaptées au jeu qu'on veut développer. Ces systèmes ont ensuite été revisités par d'autres membres du projet pour les améliorer.

À la suite de cela, un problème assez important est apparu pour moi : Python n'était plus détectable sur mon ordinateur.

Cela implique que je ne pouvais plus exécuter de code Python depuis mon ordinateur, rendant l'avancement du projet très difficile ; ainsi, d'autres membres ont continué quelques-uns des codes que j'ai commencés et ont amélioré ceux-ci. Cela a aussi engendré du retard par rapport au système de pêche dont je suis le responsable.

Heureusement, dorénavant, j'ai un nouvel ordinateur qui fonctionne correctement et je n'ai plus ce souci de présent, malgré le fait que pour le moment mon ordinateur n'arrive pas à exécuter Panda3D et donc Ursina.

### 2.3.2 Bilan Noah

Quand j'ai commencé à coder, la caméra était déjà faite et les déplacements étaient commencés. Cependant, la physique d'Ursina ne nous convenait pas. J'ai donc recréé une physique simple, basée sur les vraies lois de la physique et adaptée à nos attentes.

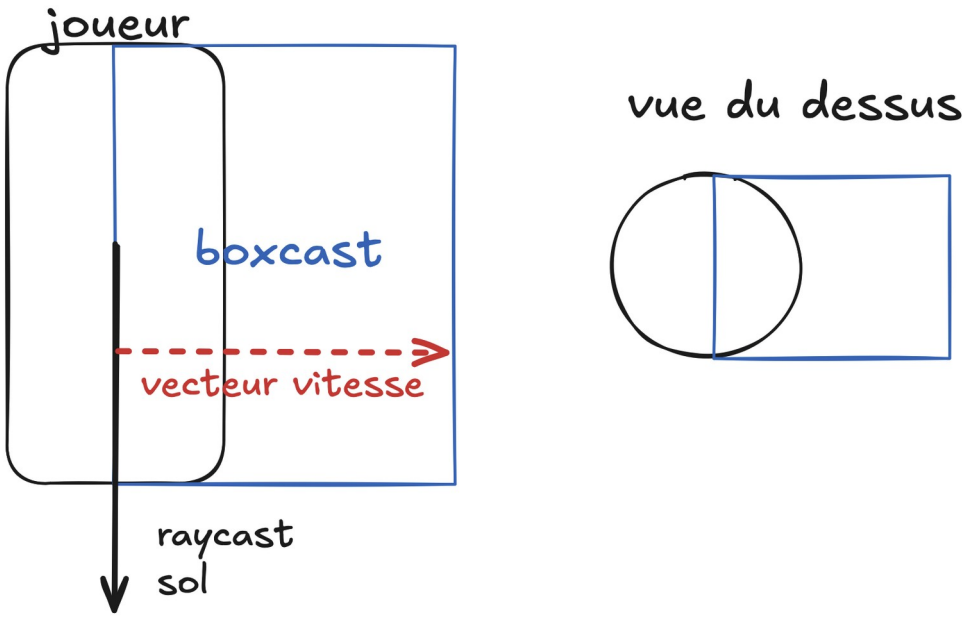
J'ai mis beaucoup de temps à coder cela, car je n'étais pas encore très à l'aise avec Ursina. Une fois la physique basique recréée, le Deputy Lead m'a aidé à finaliser les déplacements comme nous le souhaitions. Une fois les déplacements terminés, nous nous sommes attaqués aux collisions.

Comme le Deputy Lead l'explique dans sa partie, nous avons passé énormément de temps à chercher des formules mathématiques et physiques, ainsi que des fonctions d'Ursina, mais nous n'avons pas réussi.

Nous avons donc décidé avec le Deputy Lead d'utiliser les collisions de Panda3D, car elles devraient fonctionner avec le multijoueur. Je suis donc en train de découvrir et de comprendre Panda3D. Je suis également responsable de l'IA des poissons.

### 2.3.3 Bilan du Deputy Lead

Pour les mouvements, ceux déjà implémentés étaient assez rudimentaires et loin d'être parfaits. J'ai donc retravaillé cette partie pour ajouter une physique, notamment la normalisation du vecteur vitesse pour éviter d'aller plus vite en diagonale, ainsi que l'implémentation d'une accélération et d'une tentative de collision.

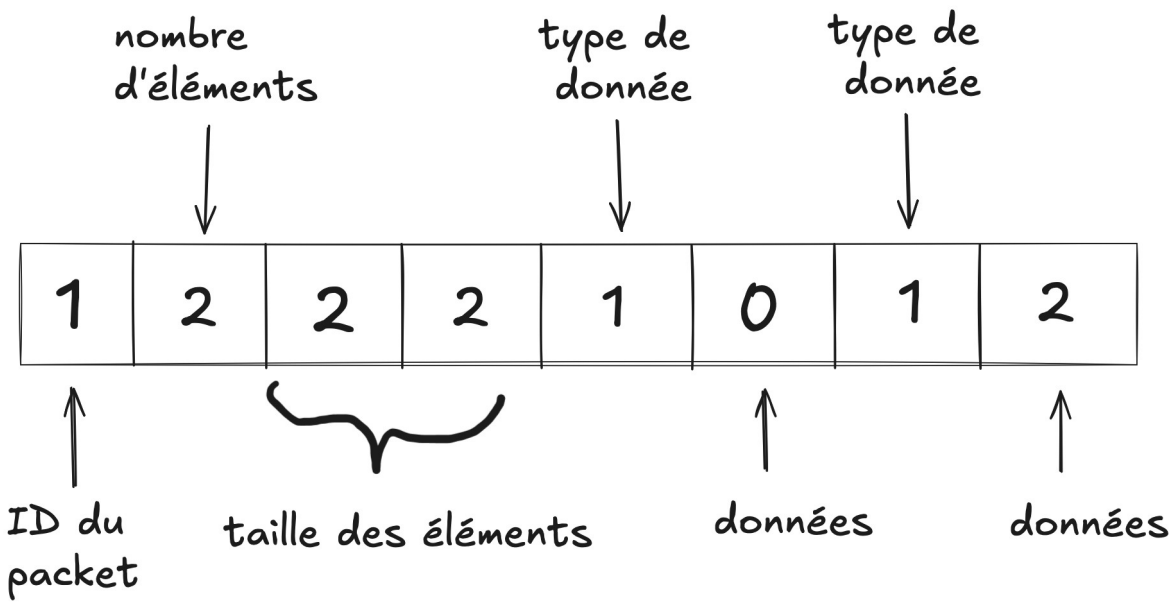



Malgré de nombreuses heures de recherche et d'essais, je n'ai malheureusement pas réussi à parfaire le système de collision, notamment sur l'axe XZ, en raison de sa complexité et du fait qu'Ursina ne propose que des fonctions très simples, peu adaptées à nos besoins. N'étant pas responsable majeur de cette partie et n'y parvenant pas, je suis passé à la partie réseau des mouvements.

## 2.4 Multijoueur

### 2.4.1 Bilan du Deputy Lead

Pour le réseau, grâce à mes expériences passées, j'avais déjà créé un jeu multijoueur, une recréation du jeu de société *Oriflamme*. J'ai donc repris une grande partie de ce code, qui implémente déjà un serveur basique permettant d'envoyer du texte directement au serveur et vice-versa. Je me suis rapidement rendu compte que ce système minimal ne tiendrait pas au fil du développement du projet. C'est pourquoi j'ai implémenté un système d'encodage et de décodage de paquets, en le rendant facile à utiliser, avec la possibilité d'enregistrer ses propres types de données grâce à un décorateur parfaitement intégré dans l'écosystème Python.




*Représentation de paquet sous forme de bytes*

La plupart des problèmes rencontrés lors du développement du réseau et ailleurs étaient liés au système d'importation de Python, qui peut varier selon l'endroit d'où l'on lance le fichier `main`, ainsi qu'au typage nécessaire pour la librairie créée pour le réseau.

C'est à ce moment-là que j'ai dû approfondir mes connaissances en Python et que j'ai découvert les librairies standard `typing` et `struct`, qui se sont révélées essentielles au fonctionnement du jeu.

Pour les mouvements, nous voulions un système sécurisé : le client n'envoie donc que les touches pressées au serveur, qui recalcule la position et la corrige si nécessaire. C'est là que je me suis heurté au plus gros problème avec Ursina : le moteur ne permet pas de calculer les collisions ni de réaliser des tracés de collision.

Face à ce problème, nous avons décidé d'utiliser Panda3D, qui pallie justement cette limitation et s'intègre parfaitement à Ursina. Malgré cette difficulté, j'ai pu me concentrer sur la transmission des informations client/serveur et sur la mise en place de son bon fonctionnement.

## 2.5 IA des poissons

### 2.5.1 Bilan du Team Lead

Concernant le développement informatique, j'ai commencé à coder l'IA des poissons. Pour l'instant, ils sont statiques et se tournent fluidement vers leur prochaine destination.

### 2.5.2 Bilan Noah

Le temps passé sur les déplacements et l'apprentissage des moteurs ne m'a pas permis de réaliser cette tâche. C'est le Team Lead qui s'est occupé de commencer l'IA des poissons.

## 2.6 Map

### 2.6.1 Bilan du Team Lead

J'ai très tôt fait des croquis de la carte du jeu. Sur papier, j'ai disposé les lacs, les chemins, les arrêts de fishobus, le magasin et le décor de manière à la rendre immersive et agréable à parcourir pour le joueur. Les arrêts de fishobus permettront au joueur de se rendre rapidement au centre de la map, là où est situé le magasin. J'ai ensuite mis cette carte au propre avec Photoshop.

Durant les vacances de Noël, j'ai modélisé le terrain avec des chaînes de montagnes, des barrières et des sapins avec Blender. Je les ai assemblés pour en faire l'environnement 3D (sans texture). Il faudra l'implémenter en jeu, ce qui est possible avec le système de terrain d'Ursina qui marche avec des *heightmaps*. Pour les objets, j'utiliserai un fichier JSON pour préciser leur emplacement.

## 2.7 Site Web

### 2.7.1 Bilan du Team Lead

Au niveau du site web, j'ai fait une maquette de la page d'accueil (bannière, pochette du jeu, bouton de téléchargement, description) sur Figma. J'ai également réalisé une carte mentale de l'architecture du site (disposition des pages et des liens pour chacun des boutons). Je l'ai ensuite rendue au Développeur B pour qu'il puisse faire de cette maquette un vrai site web.

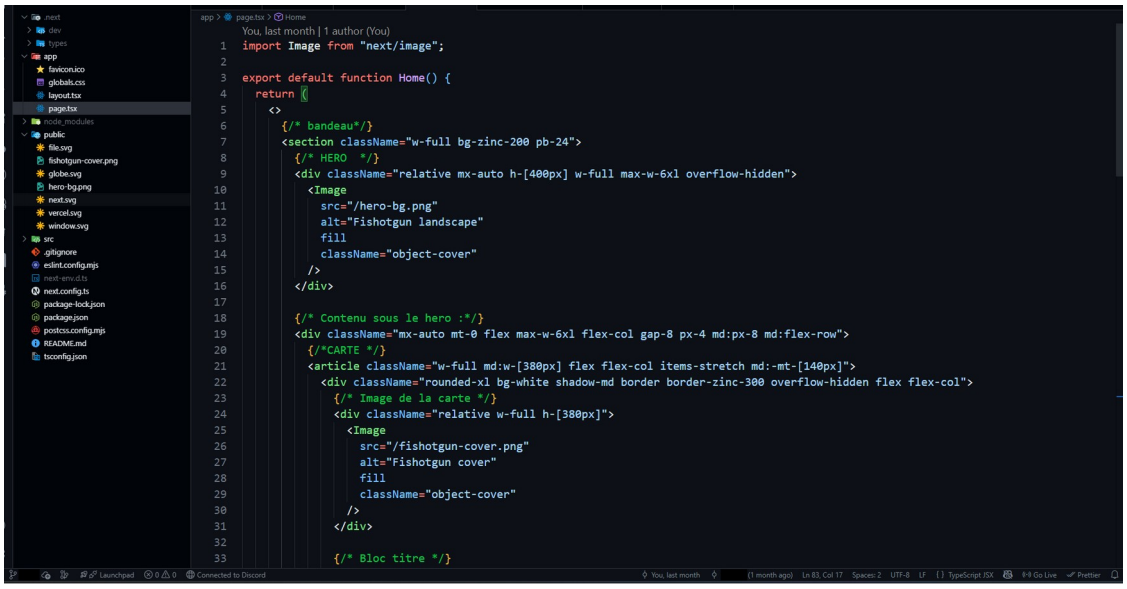
### 2.7.2 Bilan du Développeur B

Lorsque je me suis lancé dans le projet *Fishotgun*, ce que je souhaitais avant tout réaliser était le site web du jeu vidéo. Pour moi, cette tâche faisait pleinement partie du projet, dans la mesure où le site web est directement lié au jeu et à son univers, et qu'il constitue un support important pour présenter le concept, l'équipe et l'avancement du projet.

J'ai donc concentré mon travail sur la conception et le développement du site web, en réfléchissant à son architecture (HTML et CSS), à son organisation et à l'expérience utilisateur.

L'objectif était de proposer une interface claire, cohérente avec l'identité visuelle de *Fishotgun*, et pensée pour évoluer en parallèle du développement du jeu. Le jeu n'a pas encore d'identité visuelle « propre », mais tout de même, afin de ne pas partir de zéro sur la partie visuelle du site, le Team Lead m'a fourni des maquettes, ce qui m'a permis de structurer mon travail plus efficacement.

Bien que je sois à l'aise avec le développement en HTML et CSS, la partie design reste pour moi l'un des aspects les plus complexes dans la création d'un site web. Au cours du projet, j'ai toutefois rencontré une difficulté importante : j'ai été trop ambitieux dans mes choix techniques en souhaitant utiliser des frameworks avancés comme React, Next.js ou Vite, que je ne maîtrisais pas suffisamment.



```tsx
import Image from "next/image";

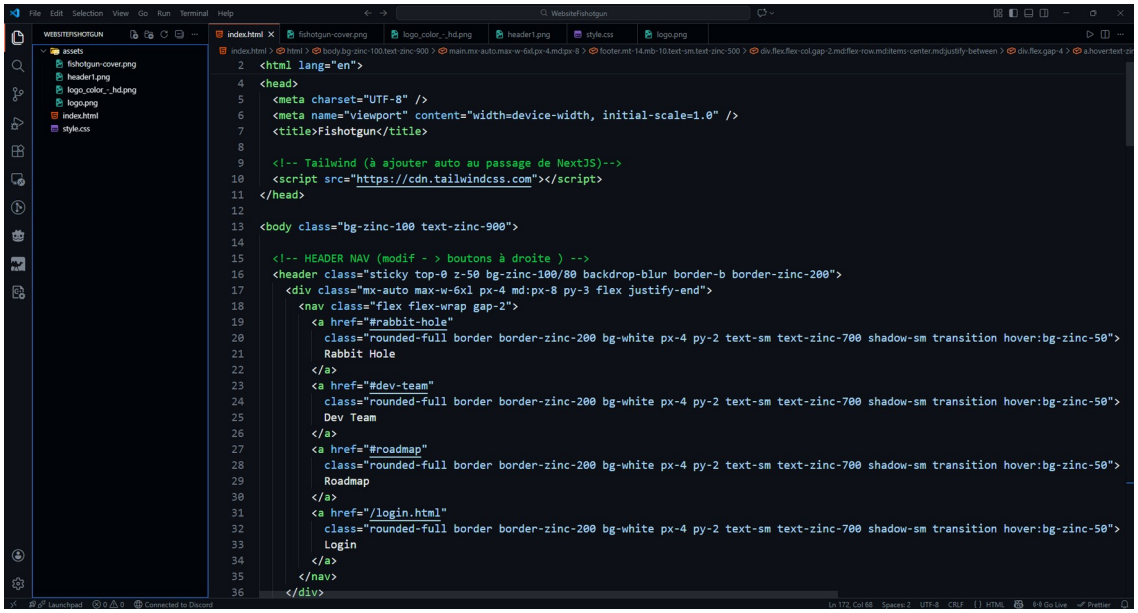
export default function Home() {
  return (
    <>
      {/* bandeau*/}
      <section className="w-full bg-zinc-200 pb-24">
        {/* HERO  */}
        <div className="relative mx-auto h-[400px] w-full max-w-6xl overflow-hidden">
          <Image
            src="/hero-bg.png"
            alt="Fishotgun landscape"
            fill
            className="object-cover"
          />
        </div>

        {/* Contenu sous le hero :*/}
        <div className="mx-auto mt-0 flex max-w-6xl flex-col gap-8 px-4 md:px-8 md:flex-row">
          {/*CARTE */}
          <article className="w-full md:w-[380px] flex flex-col items-stretch md:-mt-[140px]">
            <div className="rounded-xl bg-white shadow-md border border-zinc-300 overflow-hidden flex flex-col">
              {/* Image de la carte */}
              <div className="relative w-full h-[380px]">
                <Image
                  src="/fishotgun-cover.png"
                  alt="Fishotgun cover"
                  fill
                  className="object-cover"
                />
              </div>

              {/* Bloc titre */}
```

Cela m'a fait perdre du temps et m'a obligé à revoir mon approche. J'ai donc décidé de repartir sur une base plus simple, en utilisant uniquement un framework CSS, afin de centraliser le développement du site et de garantir une progression plus stable et maîtrisée.

Cette décision m'a permis de mieux comprendre mes limites actuelles, mais aussi d'apprendre à adapter mes choix techniques aux contraintes du projet. Le site web avance désormais de manière plus cohérente, et mes connaissances progressent progressivement.



```html
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Fishotgun</title>

  <!-- Tailwind (à ajouter auto au passage de NextJS)-->
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-zinc-100 text-zinc-900">

  <!-- HEADER NAV (modif - > boutons à droite ) -->
  <header class="sticky top-0 z-50 bg-zinc-100/80 backdrop-blur border-b border-zinc-200">
    <div class="mx-auto max-w-6xl px-4 md:px-8 py-3 flex justify-end">
      <nav class="flex flex-wrap gap-2">
        <a href="#rabbit-hole"
          class="rounded-full border border-zinc-200 bg-white px-4 py-2 text-sm text-zinc-700 shadow-sm transition hover:bg-zinc-50">
          Rabbit Hole
        </a>
        <a href="#dev-team"
          class="rounded-full border border-zinc-200 bg-white px-4 py-2 text-sm text-zinc-700 shadow-sm transition hover:bg-zinc-50">
          Dev Team
        </a>
        <a href="#roadmap"
          class="rounded-full border border-zinc-200 bg-white px-4 py-2 text-sm text-zinc-700 shadow-sm transition hover:bg-zinc-50">
          Roadmap
        </a>
        <a href="/login.html"
          class="rounded-full border border-zinc-200 bg-white px-4 py-2 text-sm text-zinc-700 shadow-sm transition hover:bg-zinc-50">
          Login
        </a>
      </nav>
    </div>
```

À terme, l'objectif est d'intégrer du JavaScript afin de rendre le site plus dynamique, puis d'établir une connexion entre le site et le jeu via une base de données, en collaboration avec l'équipe chargée du networking. L'utilisation de frameworks à l'avenir n'est pas écartée.

# 3 Avancements Soutenance 2

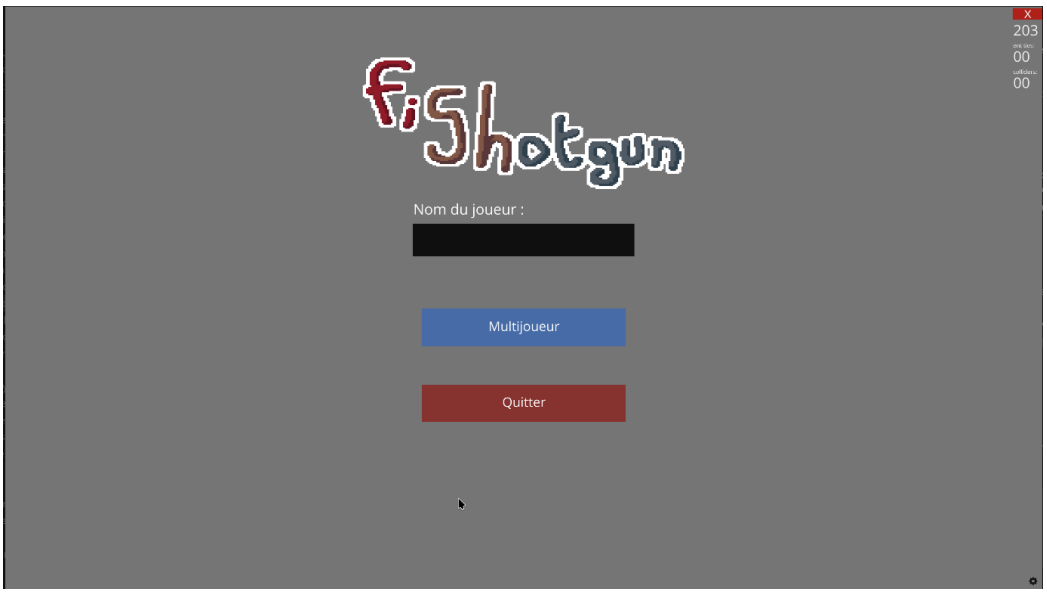
## 3.1 Menu

### 3.1.1 Bilan du Deputy Lead

J'ai avancé sur les menus avec notamment la création du système de mise en place de menu simple. J'ai mis en place une variable globale correspondant au menu qui est affiché et deux méthodes **hide** et **show** qui permettent soit de cacher les menus, soit d'afficher d'autres menus.

Chaque menu est fait à partir de la classe **Menu** à laquelle on lie les différents éléments du menu comme les boutons. Aussi j'ai créé une classe spécifique pour faire des boutons qui changent de menu afin de faciliter cette tâche.

Également, le système de menu est constitué d'un **registre** permettant d'accéder à un menu à partir de son nom. Le menu principal est constitué d'un bouton multijoueur et d'un bouton quitter et du logo du jeu avec également un Input Field pour pouvoir rentrer son nom d'utilisateur.




J'ai également dû implémenter un système de liste de boutons, car Ursina n'en propose pas par défaut. Pour cela, j'ai créé une liste de boutons placés les uns à la suite des autres, puis j'ai utilisé un shader afin de n'afficher les boutons que dans une zone précise de l'écran. Cela permet d'avoir un nombre potentiellement illimité de boutons qui peuvent être affichés dans cet espace, de manière similaire à une liste défilante.

J'ai toutefois rencontré quelques difficultés lors de cette implémentation, notamment à cause de problèmes de compatibilité avec OpenGL dus au fait que je développe sous une machine virtuelle Linux sur mon Mac qui ne supporte pas bien OpenGL.

### 3.1.2 Bilan du Développeur A

J'ai contribué au développement du menu ; ainsi, je vais partager ce que j'ai réalisé de mon côté, que le Deputy Lead a un peu modifié par la suite pour embellir notre interface.

Le système de menus repose sur une classe « Menu » dans « menu.py » qui hérite d'« Entity » d'Ursina, avec « parent=camera.ui » pour que les éléments soient toujours affichés « par-dessus » la scène 3D (devant notre écran).

Un dictionnaire global « _menus » référence tous les menus enregistrés par leur identifiant string, et deux fonctions « show() » et « hide() » gèrent l'activation et la désactivation des menus.

La fonction « show() » désactive le menu courant avant d'en activer un nouveau, et déverrouille la souris (« mouse.locked = False », « mouse.visible = True »). La fonction « hide() » fait l'inverse et reprend le jeu via « application.resume() ».

La navigation est gérée par des boutons classiques d'Ursina avec des callbacks « on_click » assignés à des méthodes. La classe « LinkingButton » hérite de « Button » et prend un menu cible en paramètre, appelant « show() » au clic.

La navigation se présente comme cela : « MainMenu » → « PlayMenu » → soit « ServerListMenu » pour rejoindre un serveur existant, soit « HostMenu » pour en créer un. Un bouton retour est présent sur chaque écran pour revenir en arrière sans se perdre dans la navigation.

Le « MainMenu » affiche le titre « FISHOTGUN » en bleu ciel ainsi qu'un champ de saisie pour entrer son pseudo. Ce champ hérite d'« InputField » d'Ursina et surcharge « update() » pour que le champ ne soit actif que quand son menu parent est affiché. Le pseudo saisi est stocké dans une variable globale « name » au moment où le joueur clique sur « Multijoueur », via une méthode « switch() » qui wrap le callback original du bouton.

Le « ServerListMenu » affiche la liste des serveurs ajoutés manuellement par le joueur via « AddServerMenu », où il entre une adresse au format « ip:port ». Les serveurs ajoutés apparaissent sous forme de « ServerButton » dans une liste scrollable ; le scroll est géré en déplaçant un « Entity » parent « button_list » verticalement, avec un clamp pour éviter que la liste parte trop loin.

Un système de shader GLSL clippe visuellement les boutons qui dépassent de la zone de liste. Un premier clic sur un serveur le sélectionne, un deuxième clic déclenche la connexion via « world.join_world() ».

Au niveau design, on n'a pas encore intégré nos textures personnalisées dans le jeu, donc on a opté pour un côté minimaliste pour le moment. Le design final consistera en un menu sous forme de carnet posé sur une table de pique-nique, avec le fusil à pompe du joueur à côté.




## 3.2 Physique

### 3.2.1 Bilan Noah

J'ai énormément travaillé sur la physique du jeu depuis la dernière soutenance. En effet, lors de la dernière soutenance, nous vous avions montré un jeu avec une physique cohérente et satisfaisante.

Cependant, nous n'avions pas abordé les collisions et, lorsque nous avons commencé à les implémenter, nous nous sommes rendu compte que la manière dont nous gérions la physique était incompatible avec celles-ci. Nous avons donc dû faire machine arrière.

Nous avons d'abord regardé s'il était possible de revenir à la première méthode, c'est-à-dire utiliser le moteur Ursina pour gérer la physique ainsi que les collisions. Après avoir beaucoup travaillé dessus, nous avons constaté qu'Ursina gérait très mal les collisions.

Avec le Deputy Lead, nous avons donc cherché une autre solution. Nous avons décidé d'utiliser Bullet, directement intégré dans Ursina, qui a la particularité de très bien gérer les collisions et la physique du jeu. Cependant, pour cela, toute la physique doit être attribuée uniquement à Bullet, sans intervention d'Ursina. Il faut donc pratiquement séparer Ursina et Bullet.

C'est, je pense, ce qui m'a pris le plus de temps. Une fois que j'ai compris comment fonctionnait Bullet, il n'était pas très difficile de créer une physique avec des collisions, notamment au sol. En revanche, l'intégrer dans notre projet et remplacer tout ce qui était géré par Ursina a été plus complexe. J'ai donc dû corriger les erreurs une par une et tout revoir.

Par exemple, une erreur que j'ai mis du temps à identifier venait du fait que certaines classes héritaient de la classe Entity. Cette classe pouvait attribuer une sorte de physique aux objets ou au personnage, ce qui créait un conflit et empêchait Bullet de fonctionner correctement.

J'ai également dû positionner notre personnage sur un bloc de terre, car la carte, qui est en cours de développement, ne possède pas encore de collisions, ce qui m'empêchait de tester mon code.

J’ai finalement réussi à terminer cette partie, mais pour l’instant, la physique et les collisions restent sur une branche différente de la branche main de notre projet. En effet, il subsiste de petits conflits entre la position estimée par le serveur et la position réelle calculée par le client lorsque la vitesse devient trop élevée, ce qui provoque des déplacements saccadés.

## 3.3 Network

### 3.3.1 Bilan du Deputy Lead

Cette partie du projet est celle qui m’a pris le plus de temps en raison de sa complexité technique. J’ai commencé par améliorer le système de parsing des paquets réseau. L’objectif était d’optimiser la taille des données échangées entre le client et le serveur.

Pour cela, j’ai ajouté la possibilité d’encoder la taille des données sur 1 ou 2 octets, en utilisant le premier bit comme indicateur (flag).

Ce mécanisme permet aux petits objets d’utiliser seulement un octet pour stocker leur taille, tandis que les objets plus volumineux peuvent en utiliser deux, ce qui permet de représenter des tailles allant jusqu’à 32 768 bits. Cette optimisation réduit la taille moyenne des paquets et améliore l’efficacité globale des communications réseau.

Après cette amélioration, je me suis concentré sur une autre partie particulièrement complexe : la synchronisation des différents joueurs dans l’environnement multijoueur. Une approche simple aurait consisté à recevoir directement la position envoyée par chaque joueur et à la retransmettre aux autres. Cependant, cette méthode pose des problèmes importants, notamment en matière de sécurité et de cohérence, car un client pourrait envoyer n’importe quelle position et se téléporter librement dans le monde du jeu.

Pour éviter cela, j’ai mis en place un système de correction de mouvement côté serveur. Le principe est que le joueur n’envoie pas directement sa position, mais uniquement ses entrées de contrôle (inputs).

Pour cela, j’ai créé une classe dédiée appelée InputField, qui utilise un masque de bits afin de représenter efficacement les différentes actions possibles du joueur. Cette approche permet de réduire la quantité de données envoyées dans les paquets tout en conservant les informations nécessaires à la simulation du mouvement.

Lorsque le serveur reçoit ces entrées, il met à jour l’état du joueur correspondant puis simule lui-même le mouvement attendu. Une fois la simulation effectuée, le serveur calcule la nouvelle position officielle du joueur et l’envoie à tous les clients connectés, y compris au joueur d’origine. De cette manière, le serveur reste l’autorité principale sur l’état du jeu.

Du côté des clients, le traitement diffère légèrement selon qu’il s’agit des autres joueurs ou du joueur local. Lorsqu’un client reçoit la position des autres joueurs, il interpole simplement leur position entre les différentes mises à jour envoyées par le serveur. Cette interpolation permet d’obtenir un mouvement visuellement fluide malgré le fait que les mises à jour réseau ne soient pas continues.

En revanche, la gestion de la correction pour le joueur local a été plus complexe à mettre en place. Le client du joueur fonctionne à la fréquence de rafraîchissement du jeu, soit environ 60 mises à jour par seconde, tandis que le serveur fonctionne à un rythme plus faible, d’environ 20 mises à jour par

seconde. À cela s'ajoute la latence réseau, qui crée un décalage supplémentaire entre la position calculée côté client et celle validée par le serveur.

Pour résoudre ce problème, j'ai mis en place un système d'interpolation entre la position attendue côté client et celle renvoyée par le serveur.

Cela permet de maintenir la synchronisation avec l'état officiel du serveur tout en évitant des corrections trop brutales qui pourraient provoquer des saccades perceptibles pour le joueur. Ainsi, les ajustements restent progressifs et le mouvement conserve une certaine fluidité.

Ce système pourrait encore être amélioré, notamment pour mieux gérer les situations de forte latence ou de perte de paquets réseau. Cependant, dans son état actuel, il offre déjà un fonctionnement satisfaisant pour les conditions d'utilisation prévues dans le projet.

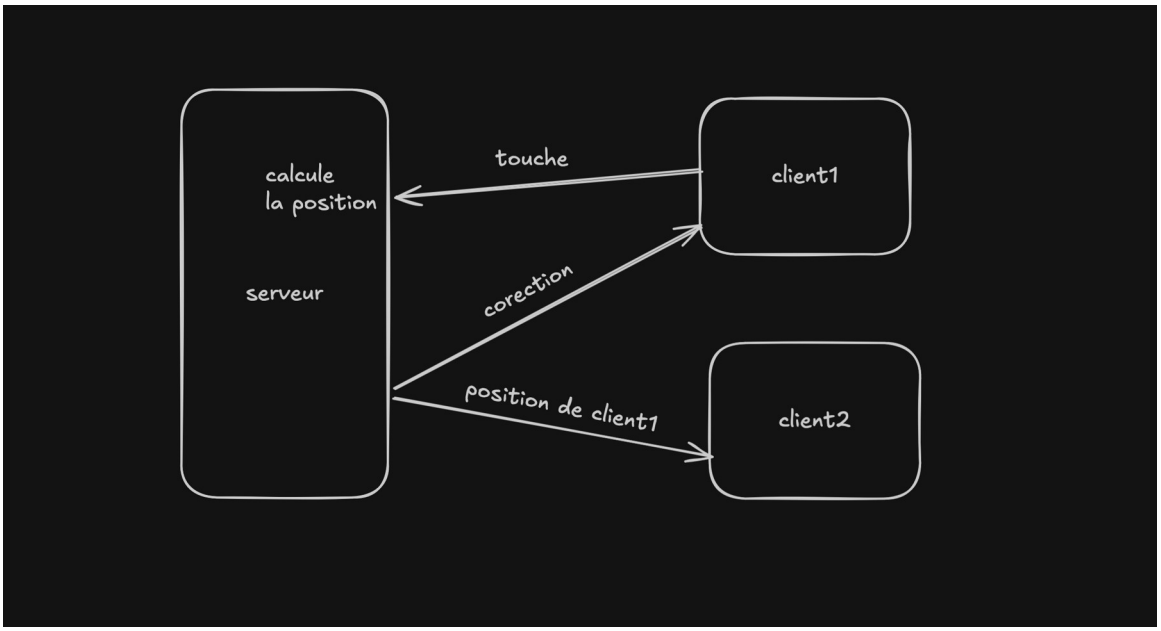



*Schéma explicatif*

## 3.4 IA des poissons

### 3.4.1 Bilan du Team Lead

Concernant l'IA des poissons, j'ai commencé par créer un environnement 3D afin de pouvoir après l'intégrer dans un environnement proche de celui en jeu. Après avoir discuté avec l'équipe de la manière dont se déplaceraient les poissons, j'ai commencé à développer un algorithme de déplacement qui fonctionne ainsi :

Le poisson se déplace à une vitesse constante dans la direction où il regarde. Un point de passage est disposé à une position aléatoire sur le plan où est le poisson. Le choix du positionnement de ce point respecte des conditions qui le forcent à rester dans le cadre visible par le joueur.

L'objectif du poisson est alors de passer par ce point. L'enchainement de ces points constituera son trajet. Pour cela, le poisson tourne vers la gauche ou la droite (en fonction de ce qui est le plus rapide) afin de s'orienter vers le point. Le poisson avance alors vers le point en suivant des trajectoires arrondies qui rendent ses mouvements authentiques et légèrement difficiles à prévoir pour le joueur.

Nous avons cependant rencontré une difficulté. Le poisson, se déplaçant et s'orientant à vitesse constante, se retrouvait parfois à graviter autour du point de passage. Cela rendait la trajectoire cyclique, simple à anticiper, et l'empêchait d'avancer vers un autre point.

Pour répondre à ce problème, nous avons modifié les mécaniques d'orientation du poisson. Le rapport vitesse de déplacement / vitesse de rotation a été ajusté pour que le poisson tourne plus vite qu'il n'avance.

Également, ce dernier détecte lorsqu'il est tourné à l'opposé du point de passage et adapte sa vitesse d'orientation en conséquence : moins le poisson est orienté vers le point, plus il tourne vite. Cela permet d'éviter les trajectoires parfaitement arrondies lorsque le poisson se retourne, toujours dans l'objectif d'obtenir un déplacement naturel.

### 3.4.2 Bilan Noah

Le Team Lead présente très bien cette partie. Je l'ai aidé sur la gestion des déplacements du poisson. C'est lui qui a eu l'idée initiale de générer des points aléatoires pour orienter le poisson. Une fois cette méthode implémentée, nous avons travaillé ensemble sur ses déplacements et avons obtenu un résultat très satisfaisant.

## 3.5 Pêche

### 3.5.1 Bilan du Développeur A

Le système de pêche a bien évolué depuis la dernière soutenance ; en effet, il s'agit dorénavant d'une base solide à laquelle on pourra ajouter les fonctionnalités finales d'ici la dernière soutenance. Il s'agit d'un mini-jeu complet intégré dans l'architecture multijoueur du projet.

Pour commencer, j'ai repris le fichier « fish.py » réalisé par Noah et le Team Lead sur l'IA des poissons afin de l'implémenter dans notre jeu.

Ainsi, j'ai modifié légèrement quelques fonctionnalités telles que la méthode « look_at(target_pos) » qui gère à la fois le déplacement et la rotation du poisson vers un point cible, pour que ce soit applicable dans l'environnement du jeu.

J'ai surtout changé la manière dont les poissons se déplacent : dorénavant, ils changent de cible vers laquelle ils nagent soit lorsqu'ils sont alignés parfaitement vers cette cible soit quand ils sont trop près de celle-ci. Cela permet de rendre le mouvement des poissons plus fluide pour le joueur tout en évitant que le poisson tourne en rond en boucle pour toujours.

Une fois la mécanique de poisson validée, j'ai intégré tout ça dans le jeu principal via deux classes : « FishingSpot » dans « spot.py » et « FishingScene » dans « fish.py ».

Le « FishingSpot » est une entité sphérique placée dans le monde 3D qui détecte la proximité du joueur dans « main.py » via la fonction « distance() » d'Ursina ; quand le joueur est à portée et appuie sur « E », la fonction « enter_fishing() » est appelée.

La « FishingScene » est une classe Python pure, sans héritage d'« Entity », ce qui veut dire qu'elle n'est pas automatiquement mise à jour par Ursina ; son « update() » est appelé manuellement depuis « main.py » à chaque frame, uniquement quand « fishing_scene.enabled » est « True ».

Ce choix architectural garantit un contrôle total sur la boucle de la nouvelle scène.

Pendant la scène de pêche, le joueur du monde extérieur est désactivé via « player.disable() » et la souris est déverrouillée, ce qui empêche toute interaction avec le monde 3D principal.

Les entités créées pendant la pêche (eau, poissons, points, barres UI) sont toutes stockées dans une liste « _entities » et détruites proprement à la fin via « destroy() » ; bien sûr, elles ne sont à aucun moment visibles pour les autres joueurs et sont mises réellement sous la carte par praticité.

Un des défis techniques a été de repositionner la caméra pour la vue de pêche sans perturber la caméra du « ThirdPersonController ».

Dans Ursina, la caméra est enfant du joueur dans la hiérarchie de scène, ce qui veut dire que modifier « camera.position » directement déplace la caméra par rapport au joueur et non dans l'environnement.

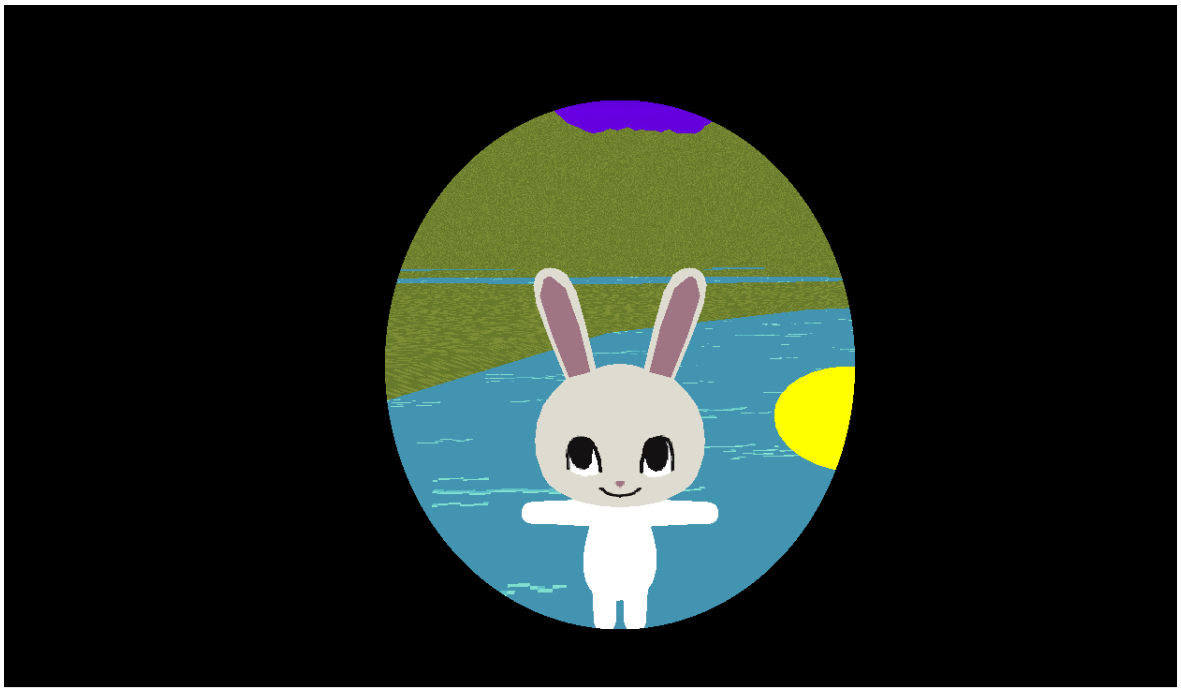
La solution a été de sauvegarder le parent original de la caméra puis de la reparenter à « scene », ce qui permet ensuite d'utiliser « camera.world_position » et « camera.world_rotation » pour la positionner librement.

La vue du dessus est obtenue avec « camera.rotation = (90, 0, 0) » et « camera.position = (0, -30, 0) », ce qui donne un décalage vertical négatif qui place toute la scène de pêche loin en dessous du monde de jeu principal, garantissant qu'aucune entité du jeu principal n'interfère visuellement.

Cette isolation est aussi importante dans le contexte multijoueur : les entités de la scène de pêche ne sont jamais envoyées au serveur, elles n'existent que localement sur la machine du joueur qui pêche.

La transition entre le monde de jeu et la scène de pêche est gérée par la classe « IrisTransition » dans « transitions.py ».

Cette classe est appelée ainsi car il s'agit d'un effet « iris wipe » inspiré des dessins animés, où un cercle noir se rétrécit progressivement jusqu'à couvrir tout l'écran, puis se rouvre.



Techniquement, cet effet est implémenté via un fragment shader GLSL appliqué à un quad plein écran attaché à « camera.ui ».

Le shader calcule pour chaque pixel sa distance au centre de l'écran et utilise « discard » pour rendre transparent tout pixel dont la distance est inférieure au rayon courant « radius », créant ainsi un cercle « troué » dans le quad noir.

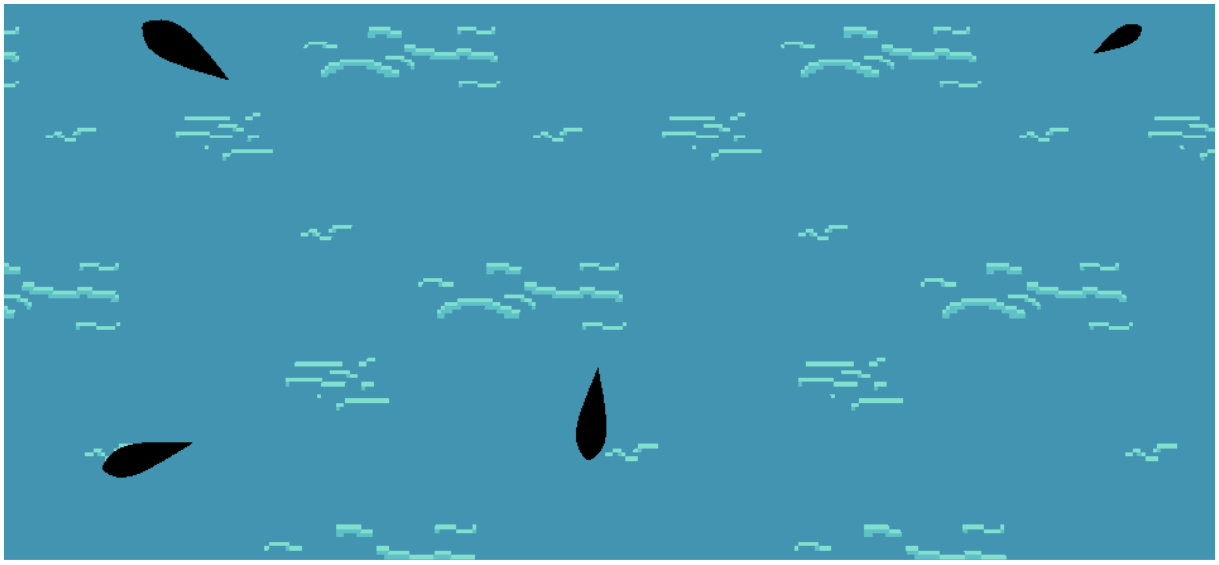
Le rayon est passé au shader comme uniform et animé en Python frame par frame, avec une correction du ratio d'aspect appliquée via « aspect/2 » pour garantir que le cercle reste rond et non ovale.

La transition se déroule en trois phases gérées par une machine d'états avec les états « CLOSING », « BLACK » et « OPENING ».

Le callback « on_black » est déclenché exactement au moment où l'iris est complètement fermé, c'est à ce moment que « fishing_scene.start() » est appelé, garantissant que le changement de caméra et de scène est invisible pour le joueur (car il se déroule pendant l'instant d'écran noir).

Au lancement de la scène, quatre poissons sont instanciés à des positions fixes avec des types choisis aléatoirement via « shuffle() ». Chaque poisson reçoit un « on_click » assigné via une lambda avec argument par défaut.

Cette syntaxe est nécessaire parce que les lambdas en Python capturent les variables par référence et non par valeur, donc sans l'argument par défaut toutes les lambdas de la boucle référenceraient le même poisson (la dernière valeur de la variable après la fin de la boucle).



Quand le joueur clique sur un poisson, « _on_fish_click » vérifie si un poisson est déjà sélectionné.

S'il ne l'est pas, « _select() » est appelé : pour chaque poisson non choisi, son point cible est détruit immédiatement et le poisson est ajouté à la liste « _fleeing » avec une direction de fuite calculée comme opposée au poisson sélectionné.

Les poissons en fuite sont mis à jour dans « update() » à une vitesse de 10 unités/seconde et détruits après 1.2 secondes via « invoke(..., delay=1.2) ». Ce délai est choisi pour que les poissons aient le temps de sortir du champ de vision avant de disparaître, donnant l'impression qu'ils s'échappent vraiment.

Trois types de poissons sont définis dans la classe « FishType » sous forme de dictionnaires Python : « WEAK » (5 PV, vitesse 1.5, taille 1.3), « NORMAL » (10 PV, vitesse 2, taille 1) et « STRONG » (20 PV, vitesse 2.5, taille 0.7). Ce sont évidemment des types temporaires de test remplaçant les types « Abondant, Discret, Insaisissable ».

Ces dictionnaires sont passés au constructeur de « Fish » via un « kwargs.pop('fish_type', FishType.NORMAL) », le « pop » est nécessaire car « fish_type » n'est pas un argument natif d'« Entity » et provoquerait une erreur si transmis à « super().**init**() ».

Les quatre poissons spawned sont toujours « [WEAK, NORMAL, NORMAL, STRONG] » pour le moment, mélangés aléatoirement, garantissant une composition équilibrée mais avec une position imprévisible.

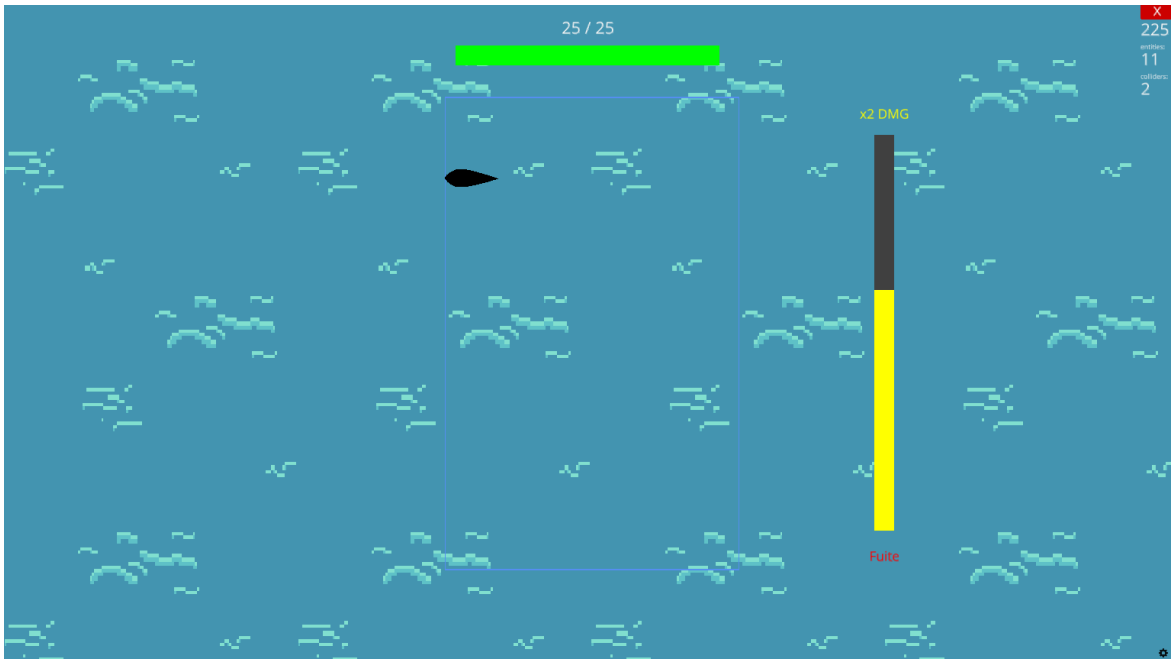
Quand le joueur clique sur le poisson sélectionné, « _deal_damage() » est appelé avec des dégâts de base de 1 (« player_damage = 1 »), doublés si la barre de pression dépasse 80 %. Quand les PV atteignent 0, « request_stop() » est appelé automatiquement et la pêche s'arrête ; le joueur reprend possession de son corps.

Deux barres d'interface ont été ajoutées après la sélection du poisson. La barre de PV est un quad horizontal en haut de l'écran avec « origin=(-0.5, 0) » pour ancrer le pivot au bord gauche, ce qui permet de réduire « scale_x » vers la droite naturellement sans recalcul de position.

De plus, sa couleur change dynamiquement : verte au-dessus de 50 % des PV, orange entre 25 % et 50 %, rouge en dessous, avec un texte qui affiche les PV numériques au format « X/max » juste au-dessus.

La barre de pression est verticale, ancrée à droite de l'écran, avec des labels fixes « x2 DMG » en haut et « Fuite » en bas. Sa hauteur est mise à jour à chaque frame en fonction de « _pressure », une valeur entre 0 et 1.

Elle monte à +0.4/seconde quand la souris est à moins de 1.5 unités du poisson dans l'espace monde, détecté via « mouse.world_point » qui raycaste sur le collider de l'eau, et descend à -0.6/seconde sinon.




Cette asymétrie intentionnelle crée une tension et du stress pour le joueur, surtout contre un poisson fort : il faut maintenir activement la souris sur le poisson pour garder la pression haute. Un flag « _stopping » a été ajouté pour éviter que « request_stop() » soit appelé plusieurs fois dans la même frame, problème qui faisait que tout disparaissait avant que l'iris puisse se refermer.

Cela survenait car « update() » continuait à s'exécuter après le premier appel ; le flag bloque immédiatement toute logique supplémentaire via un « return » en début de « update() ».

## 3.6 Assets

### 3.6.1 Bilan du Team Lead

Comme vous pourrez le voir sur le tableau de déroulement page 21, nous avons bien avancé sur la création d'assets pour le jeu. La création de différentes textures constitue en grande partie cet avancement.

En utilisant Photoshop, j'ai pu faire des textures pour : le bois, l'eau, l'herbe, le menu… Ces textures sont visibles en annexe de ce rapport. Elles ont été faites et pensées en équipe, que ce soit en appel avec le Deputy Lead ou abordées en réunion comme pour le menu.

Cela nous a également permis de pouvoir pointer plus distinctement la direction artistique choisie pour le jeu.

Également, pour ce qui est de la partie 3D, j'ai pu travailler sur la conception et l'application d'une texture sur le modèle du joueur. Je me suis ensuite directement attelé à créer un modèle pour le fusil à pompe, élément central de notre jeu. Toujours en utilisant Blender, j'ai pu concevoir le modèle en m'appuyant sur des images de référence et lui apposer une texture que j'ai faite.

Ce modèle a déjà été utilisé, notamment dans la scène de menu de démarrage comme il est possible de le constater dans la partie 2.1.2 du Développeur A. Il reste encore à associer le modèle du joueur avec celui du fusil.

Le Deputy Lead s'est occupé d'appliquer les textures sur le modèle du terrain.

Enfin, concernant l'environnement sonore, rien n'a été ajouté. Les musiques ont été finies avant la première soutenance. Quant aux bruitages et autres effets sonores, nous travaillerons dessus dans le but de rendre l'expérience de jeu la plus immersive possible.

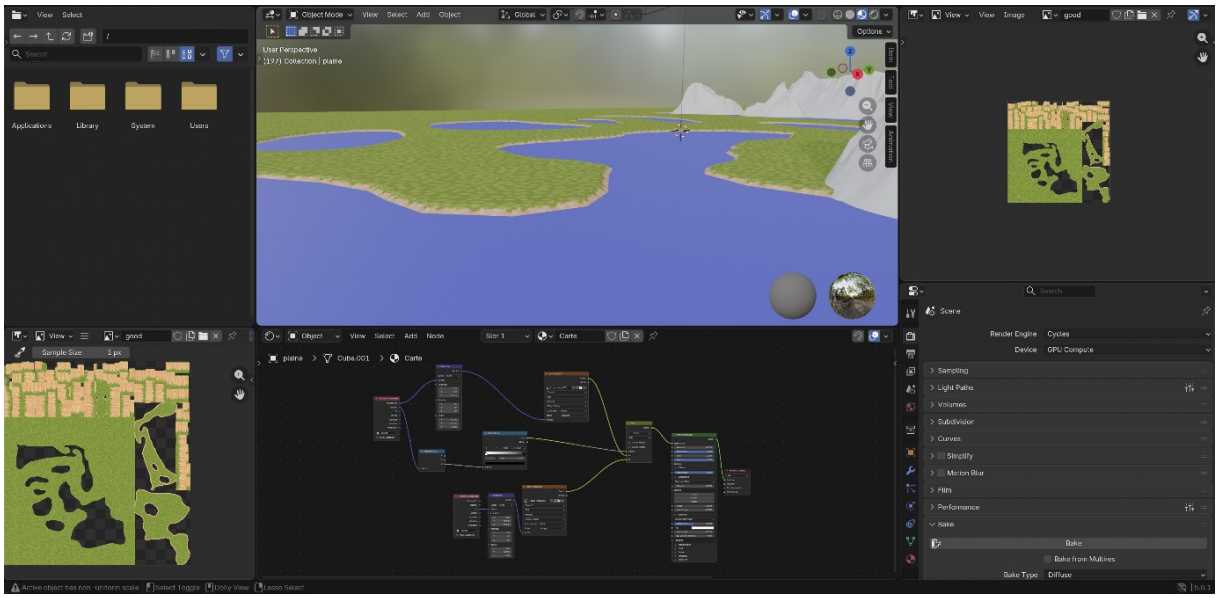
### 3.6.2 Bilan du Deputy Lead

Je me suis ensuite occupé de la partie *world* du jeu. La première étape a consisté à intégrer directement dans le jeu, avec Ursina, le modèle 3D du terrain réalisé par le Team Lead ainsi que ses textures de base. L'objectif était d'avoir rapidement une première version fonctionnelle du monde en jeu, afin de vérifier que le modèle s'affichait correctement et que les textures principales étaient bien appliquées sur le terrain.

Une fois cette intégration réalisée, je me suis concentré sur l'amélioration du rendu visuel et sur la préparation des textures dans Blender.

J'ai notamment travaillé sur l'UV mapping du terrain afin d'avoir un meilleur contrôle sur la manière dont les textures sont appliquées sur la géométrie. Cette étape est importante pour éviter les déformations de texture et obtenir un rendu plus propre, même si cette version retravaillée n'est pas encore entièrement importée dans le jeu.

Comme on peut le voir sur l'image, le matériau est composé de plusieurs nodes qui permettent de mélanger les textures et de contrôler leur affichage sur le terrain.



Le principe consiste à utiliser deux textures principales : une texture d'herbe pour le sol et une texture différente, notamment du sable, pour les zones de transition comme les bords proches de l'eau ou certaines pentes du terrain.

Pour déterminer quelle texture doit être affichée, j'utilise la normale de la surface du terrain. Cela permet d'adapter automatiquement la texture en fonction de l'orientation des faces.

J'ai également ajusté l'échelle des textures afin que le motif reste cohérent visuellement et ne paraisse pas trop répétitif sur les grandes surfaces.

Cependant, une contrainte technique s'est posée : le moteur Ursina ne supporte pas facilement l'utilisation de plusieurs textures combinées dans un même matériau. Il a donc fallu trouver une solution pour conserver le rendu tout en restant compatible avec le moteur.

Après m'être renseigné, j'ai utilisé une fonctionnalité de Blender appelée *baking*. Pour cela, j'ai créé une nouvelle UV map dédiée au terrain, puis j'ai généré une texture finale à partir du rendu complet du matériau. Concrètement, Blender calcule le résultat du mélange des différentes textures et l'enregistre dans une seule image. Cette texture peut ensuite être utilisée directement dans le jeu.

Cette méthode correspondait bien à mon besoin, car elle permet de garder le rendu visuel préparé dans Blender tout en utilisant une seule texture dans Ursina.

## 3.7 Site Web

### 3.7.1 Bilan du Développeur B

Pour ma part, je me suis occupé du site vitrine de Fishotgun.

J'ai utilisé une stack web simple en trois parties.

D'abord, HTML, qui sert à faire la structure du site. C'est ce qui permet d'organiser les différentes sections, comme la bannière, la présentation du jeu, l'équipe ou la roadmap.

Ensuite, TailwindCSS, qui sert à gérer l'apparence du site. C'est grâce à ça que j'ai pu travailler la mise en page, les couleurs, les espacements, les cartes, les effets visuels et l'ambiance générale du site.

Enfin, JavaScript, qui sert à rendre le site interactif. Par exemple, c'est ce qui permet de gérer le changement de langue, le lecteur de musique, le carrousel d'images, ou encore les effets au survol dans la section équipe.

Donc, pour résumer très simplement :

- HTML = la structure
- TailwindCSS = le style
- JavaScript = l'interactivité

À partir de cette base, j'ai retravaillé le site pour qu'il soit plus lisible, plus cohérent visuellement, et plus représentatif de l'univers de Fishotgun. J'ai aussi amélioré les textes, surtout en français, et réorganisé le code JavaScript en plusieurs fichiers pour qu'il soit plus propre et plus facile à maintenir.

## 3.8 Tchat

### 3.8.1 Bilan du Deputy Lead

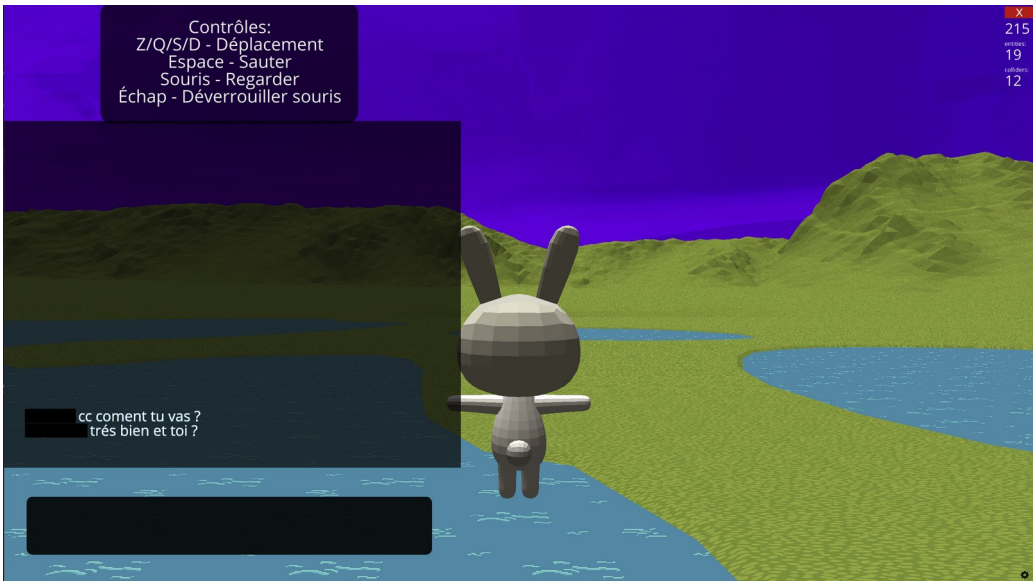
Je me suis chargé de la mise en place du tchat. J'ai d'abord implémenté la logique côté serveur : le joueur envoie un paquet contenant son message au serveur, et celui-ci se contente ensuite de diffuser ce message à l'ensemble des autres joueurs.

La partie la plus complexe a été la conception du menu de chat lui-même, notamment en raison du défi que représentait la création d'une interface personnalisée avec Ursina.

Pour l'affichage des différents messages, j'ai mis en place un fond gris sur lequel vient se superposer une liste de messages. Ceux-ci sont affichés en utilisant la classe Text d'Ursina, avec les noms des joueurs colorés en orange pour une meilleure lisibilité.

Concernant la saisie de texte, j'ai réutilisé le composant InputField d'Ursina. Je l'ai adapté afin de n'afficher qu'une partie du texte complet en surchargeant la méthode render. Cela permet d'écrire des messages longs sans qu'ils ne dépassent visuellement le cadre défini.

J'ai toutefois rencontré certaines complications, notamment avec l'affichage du texte dans Ursina, qui applique un contour de manière irrégulière. Ne pouvant pas réellement modifier le moteur et n'ayant pas trouvé de solution satisfaisante à ce problème, j'ai choisi de poursuivre le développement sur d'autres aspects du projet.




*Rendu du chat*

# 4 Futures tâches

## 4.1 Menu

### 4.1.1 Bilan du Deputy Lead

Le menu marche correctement, nous allons surtout implémenter nos assets personnalisés dans celui-ci dans un futur proche.

Ainsi, on va devoir trouver une solution pour que le menu marche en « 3D », car le carnet posé sur la table le sera. Soit nous ferons un faux carnet invisible qui sera en 2D sur l'UI, pour faciliter les clics, soit à l'aide d'un raycast de la souris dans l'environnement ; on réfléchit encore à laquelle de ces deux solutions sera la meilleure.

## 4.2 Pêche

### 4.2.1 Bilan du Développeur A

Pour ce qui est de la pêche, la mise en place d'un système de collection du poisson est en cours, afin de permettre au joueur de récupérer le poisson après l'avoir battu ou le perdre lors d'une fuite.

De plus, plusieurs fonctionnalités seront ajoutées telles que les types/raretés de poissons, qui vont modifier leurs statistiques permettant un gameplay varié.

Ces poissons pourront ensuite être vendus pour un certain prix en boutique ; ainsi, il faudra leur donner un multiplicateur d'argent en fonction de leur taille aussi.

## 4.3 Inventaire

### 4.3.1 Bilan du Développeur A

L'inventaire n'est actuellement pas mis en place, mais ce sera un de nos plus gros défis prochainement étant donné qu'il y a beaucoup de code derrière celui-ci.

Ainsi, on compte réaliser un inventaire qui servira à stocker et montrer les poissons capturés par le joueur, et afficher le prix qu'ils vaudront lors de leur vente.

On réfléchit à la mise en place d'informations par rapport aux poissons dans l'inventaire, mais il sera probablement plus pratique d'afficher seulement leur prix dans l'inventaire quand on passe la souris dessus, et afficher les vraies statistiques des poissons dans le Fishodex qu'on va bientôt mettre en place.

## 4.4 Boutiques

### 4.4.1 Bilan Noah

Nous n'avons pas pu commencer l'implémentation de la boutique qui était prévue. Nous comptons créer une boutique proposant des améliorations de notre gameplay, ainsi que des achats possibles,

notamment des munitions ou un nouveau shotgun. Nous sommes donc en retard sur ce point, car, pour ma part, la physique m'a pris beaucoup plus de temps que prévu.

## 4.5 Organisation

Nous avons pu constater un retard dans l'avancement du projet, en partie dû à un suivi des tâches imprécis, ce qui a ralenti la production. De plus, de nombreux problèmes imprévus se sont présentés au cours du développement, entraînant des ajustements constants et ralentissant davantage la progression et la mise en production du projet.

Beaucoup de problèmes ainsi qu'une méconnaissance du moteur ont également fait que beaucoup de temps a été investi dans la recherche et la correction plutôt que dans l'ajout pur de fonctionnalités. Lors de notre dernière réunion, nous avons justement revu nos objectifs à la baisse afin de pouvoir livrer un projet fini dans les temps, en nous concentrant sur la pêche et les fonctionnalités principales du jeu.

# 5 Avancements Soutenance 3

## 5.1 Bilan UI

### 5.1.1 Bilan du Deputy Lead

Lors de la dernière soutenance, il avait été demandé d'ajouter davantage d'éléments graphiques au menu principal, notamment avec la mise en place d'un carnet dont les pages se déplacent entre les différents menus. Pour cela, on a utilisé les différentes textures de carnet réalisées par le Team Lead. On en a intégré deux : une première servant à effectuer les animations de rotation des pages, et une seconde permettant de donner l'illusion d'un véritable carnet avec plusieurs pages. À la fin de l'animation, lorsque la page a terminé sa rotation, le nouveau menu sélectionné est alors affiché.




Afin de gérer ce système, on a créé une nouvelle classe dédiée à l'affichage des pages et de l'arrière-plan du menu principal. On s'est également occupé d'intégrer la nouvelle police d'écriture réalisée par le Team Lead sur les différents boutons des menus afin d'obtenir l'aspect graphique souhaité. On a aussi ajouté, dans le menu des serveurs, la possibilité de supprimer les serveurs enregistrés.

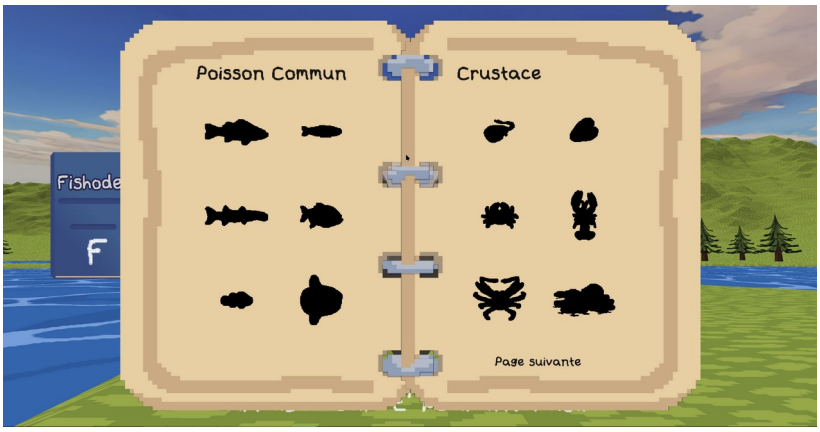



Une fois le menu principal terminé, je me suis attaqué au menu le plus important du jeu : le Fishodex. Celui-ci permet de répertorier tous les poissons rencontrés et de les centraliser dans une interface unique. Pour sa conception, j'ai repris l'apparence du menu principal avec les mêmes textures de pages, mais cette fois sous la forme d'un carnet double au lieu d'une seule page rotative. Ici,

l'intégralité du menu est affichée en une seule fois, avec des poissons visibles sur les deux pages centrales.

La réalisation de ce système a été assez complexe, principalement à cause de la gestion des zones de collision et des distances dans Ursina. J'ai notamment dû masquer certaines zones de collision lorsque les pages se tournent, car elles ne suivaient pas visuellement l'animation des pages.

Concernant les poissons du Fishodex, j'utilise un registre partagé entre le client et le serveur contenant toutes les informations des poissons existants. En fonction de leur index, je récupère leur nom, leur identifiant, leur rareté ainsi que leur description. L'identifiant me permet de charger l'image correspondante, puis d'afficher le nom et la description du poisson lorsque le curseur passe dessus, à condition qu'il soit débloqué. Si le poisson n'a pas encore été découvert, seule sa silhouette est visible.




On a également dû mettre en place la synchronisation du Fishodex entre le client et le serveur tout en empêchant les joueurs de pouvoir s'attribuer eux-mêmes des poissons. Pour cela, le serveur est devenu la seule source de vérité, notamment grâce à l'ajout de nouveaux paquets réseau dédiés à cette synchronisation. On a aussi fait évoluer le Fishodex afin qu'il ne soit plus uniquement un index des poissons découverts, mais également un inventaire regroupant les poissons pêchés avant leur vente.

On s'est également occupé de l'affichage de l'argent du joueur avec la création d'une interface en jeu permettant d'indiquer son montant en temps réel.

Enfin, on a corrigé de nombreux bugs, certains liés à des erreurs de conception, comme la possibilité de cliquer sur le bouton « Rejoindre » plusieurs fois, ce qui faisait apparaître plusieurs instances du joueur. On a aussi dû résoudre différents problèmes provenant directement d'Ursina, notamment des soucis d'interface où certains textes ne s'affichaient pas correctement devant les boutons ou d'autres éléments visuels.

On n'a malheureusement pas réussi à corriger le problème du curseur dans les champs de saisie, qui ne se positionne pas correctement à la fin du texte. La correction aurait nécessité une réécriture complète de la classe concernée, ce qui demandait beaucoup trop de temps pour un résultat relativement mineur.

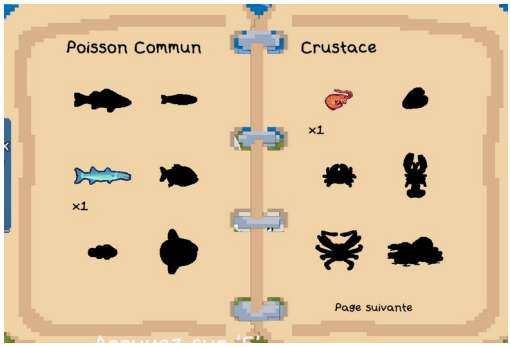
### 5.1.2 Bilan du Développeur A

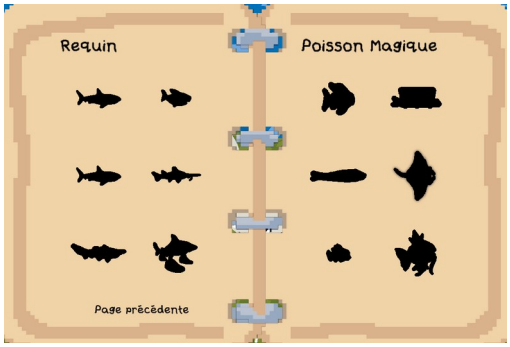
Le Fishodex est bien plus qu'une simple galerie : c'est le pilier de la collection et du suivi de progression du joueur.




Nous avons mis en place une gestion complète de l'inventaire côté serveur, garantissant que chaque donnée, du nombre exact de poissons capturés jusqu'à l'état de déverrouillage de chaque espèce, soit sauvegardée de manière persistante et sécurisée.

L'interface a été conçue pour être aussi visuelle que gratifiante. L'affichage s'adapte dynamiquement à la rareté de chaque spécimen grâce à un code couleur spécifique, permettant au joueur d'identifier en un coup d'œil ses prises les plus précieuses.




Pour donner vie à cet index, nous avons intégré des animations d'ouverture fluides qui rendent la navigation agréable, complétées par des assets graphiques des poissons minutieusement intégrés par le Team Lead.

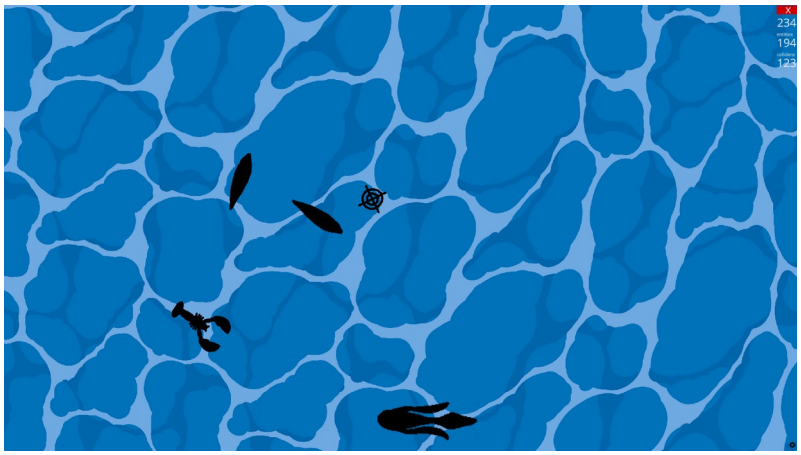
Enfin, pour une immersion totale, nous avons ajouté des éléments d'UI dédiés directement sur le HUD, permettant au joueur de garder un œil sur sa progression sans jamais briser le rythme de ses sessions de pêche. C'est l'outil indispensable pour tout pêcheur souhaitant compléter sa collection et mesurer l'étendue de ses exploits.

## 5.2 Pêche

### 5.2.1 Bilan du Développeur A

La refonte de la pêche a été l'un des chantiers les plus exigeants du projet. Notre objectif était de transformer une simple action utilitaire en une expérience réellement satisfaisante.

Nous avons commencé par le cœur visuel : un shader d'eau personnalisé, couplé à un effet de parallaxe dynamique. Cela donne au joueur une véritable impression de profondeur et de « visée » lorsqu'il traque ses proies sous la surface.

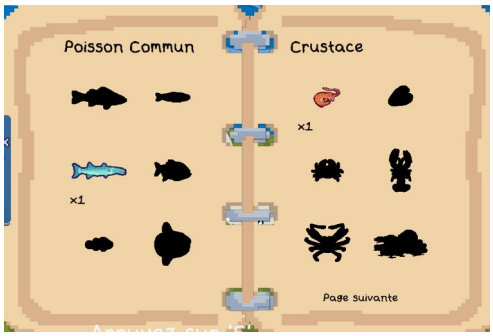


Pour renforcer ce sentiment de « chasse », nous avons troqué le curseur classique contre un réticule sur mesure, renforçant l'identité « fusil à pompe » du jeu.

Chaque tir est désormais une expérience complète : le son du coup de feu vient ponctuer l'action, tandis que chaque impact déclenche une animation de splash dynamique directement à la position du poisson.

Ce feedback est immédiat, viscéral, et les lerps que nous avons implémentés assurent que chaque transition, du recul du tir à la dissipation du splash, reste fluide et nerveuse.

La progression est désormais une réalité concrète : les poissons capturés sont ajoutés au Fishodex, avec une mise en scène dédiée pour célébrer chaque prise. Le système de spawn génère désormais quatre silhouettes, déclinées en plusieurs tailles, représentant les catégories (Communs, Crustacés, Requins, Magiques) et la rareté (Abondant, Discret, Insaisissable). Nous avons même ajouté des subtilités visuelles, comme le poisson qui s'assombrit ou change d'échelle lors de l'interaction, pour que le joueur sente physiquement la résistance de sa proie.




Enfin, concernant l'interface, la gestion de la barre de vie et de la pression a été pensée pour une lecture rapide en pleine action. Si un polissage graphique supplémentaire aurait pu apporter un peu plus de cachet, l'aspect fonctionnel est parfaitement rodé. Nous avons préféré consacrer notre temps à rendre le « feeling » du tir le plus grisant possible, et au vu du résultat, nous ne regrettons pas ce choix de priorité.

## 5.3 Shop

### 5.3.1 Bilan du Développeur A

L'économie du jeu repose désormais sur un système de boutique complet et totalement fonctionnel.

Nous avons implémenté une mécanique de dialogue contextuel : dès que le joueur s'approche du vendeur, une interface simple et intuitive s'ouvre, lui permettant d'interagir directement avec le PNJ.

Au cœur de cette boutique, nous avons intégré une gestion rigoureuse des transactions : le système permet au joueur de vider son inventaire en vendant la totalité de ses poissons pêchés en un seul clic.




Pour assurer l'intégrité et l'équité du jeu, chaque opération (qu'il s'agisse de la vente ou de l'achat) est minutieusement vérifiée côté serveur, empêchant toute manipulation des données.

Nous avons également mis en place un système de progression pour l'équipement : le joueur peut investir ses écailles pour améliorer son fusil à pompe. Chaque niveau acheté augmente la puissance de feu de 5 points de dégâts, avec une progression plafonnée au niveau 10 pour garantir un équilibrage optimal de la difficulté.




Cette boutique n'est donc pas qu'un simple lieu d'échange, mais un véritable levier de progression qui incite le joueur à retourner pêcher pour optimiser son arsenal.

### 5.3.2 Bilan du Deputy Lead

Pour le magasin, je me suis occupé de toute la partie serveur afin de sécuriser les différentes interactions du joueur avec le marchand. Le but était de vérifier que le joueur possède réellement la somme d'argent nécessaire, qu'il se trouve bien à proximité du marchand lors d'une vente, et que la vente de ses poissons lui rapporte correctement le montant prévu.

Pour cela, j'ai créé plusieurs messages réseau permettant au serveur de modifier le niveau ainsi que l'argent du joueur. De son côté, le client peut envoyer des requêtes afin de demander une amélioration de niveau ou la vente de poissons. Toutes les vérifications importantes sont cependant réalisées côté serveur afin d'éviter toute triche ou modification des données par le joueur.

J'ai également mis en place un système permettant de déterminer la valeur marchande des différents poissons en fonction de leur difficulté de capture. Ainsi, plus un poisson est rare ou difficile à attraper, plus sa valeur de revente est élevée.

## 5.4 Sauvegarde

### 5.4.1 Bilan du Deputy Lead

Après la dernière soutenance, on a commencé par la sauvegarde des données. Pour cela, on a choisi d'utiliser des fichiers binaires, car ils permettent de stocker les données de manière optimisée tout en offrant de bonnes performances de lecture et d'écriture. On a également pu réutiliser des algorithmes de conversion en octets déjà utilisés dans le système de sérialisation des paquets de données du mode multijoueur.

Côté client, toutes les données sont stockées dans un fichier nommé data.dat. Celui-ci contient le dernier pseudonyme renseigné par le joueur ainsi que la liste des différents serveurs enregistrés.

Du côté du serveur, un dossier data contient plusieurs fichiers, avec un fichier distinct pour chaque joueur. Ces fichiers enregistrent les dernières coordonnées du joueur, les poissons débloqués, l'inventaire ainsi que son niveau. Toutes ces informations sont ensuite retransmises au client lorsqu'il rejoint un serveur grâce à un paquet de données dédié.

Pour rendre ce système plus propre et plus simple à maintenir, on a ajouté dans la classe Player deux méthodes nommées save et load, permettant respectivement de sauvegarder et de charger automatiquement les données d'un joueur.

## 5.5 Animations

### 5.5.1 Bilan du Développeur A

Pour insuffler de la vie à l'interface et aux interactions, nous avons multiplié les animations dynamiques, en utilisant principalement des interpolations (*lerps*) pour garantir une fluidité constante.

L'expérience utilisateur est marquée par une transition en « iris » personnalisée, fluide et immersive, qui encadre les phases de chargement du serveur et les sorties de session.

Nous avons également soigné chaque retour visuel : l'ouverture et la fermeture du Fishodex et du menu en jeu sont désormais animées pour offrir une sensation de légèreté.

Le processus de capture de poisson a fait l'objet d'un soin particulier : dès qu'une prise est validée, une icône dédiée s'anime, accompagnée d'un texte dont la couleur s'adapte dynamiquement à la rareté du spécimen, renforçant immédiatement l'aspect gratifiant de la découverte.




Côté gameplay pur, le réticule de pêche ne se contente pas de suivre le curseur : il est animé par une rotation continue, ajoutant cette touche de tension nécessaire lors de la traque sous-marine.

Enfin, nous avons intégré un PNJ dédié au commerce : le vendeur d'écailles. Bien plus qu'un simple point d'échange monétaire, ce personnage intègre une animation d'état de repos (*idle*) qui lui confère une présence naturelle et cohérente dans l'environnement du jeu.

Chaque animation a été pensée pour que l'interface ne soit pas qu'un simple outil, mais une extension vivante de l'univers que nous avons créé.

## 5.6 Assets

### 5.6.1 Bilan du Développeur A

La carte a été pensée pour être à la fois un terrain de jeu fonctionnel et un espace immersif. Nous avons intégré un point d'intérêt central : le Magasin.

Ce lieu n'est pas qu'un simple bâtiment, c'est le cœur économique du jeu où le joueur interagit avec notre PNJ vendeur pour transformer ses prises en écailles ou investir dans des améliorations stratégiques.




Pour renforcer l'identité visuelle de notre univers, nous avons enrichi l'environnement avec un placement réfléchi de la végétation, notamment des arbres qui viennent habiller les zones et briser la monotonie du terrain. Surtout, nous avons revu la disposition des points de pêche autour des différents lacs.




Cette répartition stratégique n'est pas seulement esthétique : elle offre une véritable liberté de mouvement au joueur, lui permettant de choisir son spot de prédilection et de varier son expérience de jeu en explorant les différents recoins de la carte selon ses envies.

### 5.6.2 Bilan du Team Lead

#### *5.6.2.1 Assets 3D*

Pour créer un environnement immersif, je me suis attelé à la création de modèles 3D. Pour cela, j'ai utilisé Blender, un logiciel libre.

Pour l'environnement, j'ai commencé par travailler sur les différents objets statiques. Afin d'enrichir le décor du jeu, j'ai entièrement remodelé les arbres. J'ai donné du volume à chaque branche afin de rendre le modèle final plus agréable visuellement et moins froid.



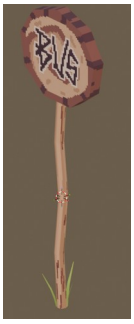
*Nouveau modèle*



*Ancien modèle*

Dans l'idée d'offrir au joueur la possibilité de se téléporter vers le centre de la carte depuis différents endroits, j'ai également modélisé un arrêt de bus. Celui-ci est entièrement réalisé en bois afin de rester cohérent avec l'esthétique globale du jeu, qui se veut proche de la nature et de l'artisanat.




L'objectif avec ces deux modèles était de leur donner un aspect imparfait. J'ai ajouté de légères déformations au pied du panneau ainsi que des écarts plus ou moins importants entre les branches de l'arbre. Cela permet d'apporter au rendu final une apparence « faite main », adaptée à l'univers enfantin de *Fishotgun*.

Le magasin étant le centre de la carte, comme l'a très bien décrit le Développeur A, j'ai souhaité rendre son apparence la plus détaillée possible. J'ai commencé par concevoir une cabane composée de quatre murs, d'un toit et d'une ouverture, en utilisant des bûches. J'y ai ensuite accroché une enseigne afin d'indiquer au joueur qu'il s'agit du magasin. Pour rendre cet espace le plus vivant et immersif possible, j'ai ajouté plusieurs accessoires décoratifs. Pour replacer cela dans le contexte du jeu, le magasin

permet au joueur de vendre ses poissons et d'améliorer son arme. J'ai donc intégré des modèles de poissons, deux fusils ainsi que des cartouches.




Après cela, j'ai ajouté au magasin son représentant principal : le gardien de la boutique. Dans l'univers du jeu, ce personnage mange les poissons que lui apporte le joueur avant de lui laisser les écailles, qui lui servent de monnaie. Nous avons donc trouvé logique de représenter ce personnage sous les traits d'un chat. Étant un mélange entre un chef cuisinier et un marchand, je lui ai ajouté un tablier ainsi qu'une petite toque. Pour le choix de ces vêtements, je me suis également inspiré du marchand de *The Legend of Zelda : Skyward Sword*.

Le vendeur est un personnage sympathique, souriant en toute circonstance, une présence réconfortante qui nous a beaucoup aidés à avancer sur le projet.

Concernant le personnage principal, aucune modification n'a été apportée au modèle 3D.

#### *5.6.2.2 Assets 2D*



L'ensemble des textures étant globalement terminé, j'ai uniquement créé une texture de pierre appliquée sur les montagnes. Malheureusement, en raison de problèmes de liaison entre les textures et les modèles dans Blender, cette texture n'a finalement pas pu être intégrée au jeu. Néanmoins, nous avons pu l'utiliser pour les différents rendus visuels du projet.

Pour la création de l'ensemble des éléments graphiques en 2D, j'ai utilisé Adobe *Photoshop*.

La principale catégorie d'éléments graphiques manquante jusqu'à présent concernait les différents poissons. J'ai ainsi créé les ombres utilisées lors des combats et dessiné un à un les vingt-quatre poissons collectionnables du jeu (voir annexe).

J'ai également réalisé l'icône des écailles du *Fishodex* destinée à l'interface du joueur. J'ai aussi conçu les différentes textures associées aux modèles 3D.

Enfin, j'ai ajouté une petite animation d'impact lorsque le joueur tire dans l'eau. J'ai réalisé cette animation en pixel art sur le site *Piskel*.



### 5.6.3 Bilan du Deputy Lead

On s'est occupé des différentes animations des entités. Pour cela, on a dû recréer entièrement l'armature du personnage, car celle d'origine présentait plusieurs problèmes, notamment au niveau de l'influence des os sur le modèle. Certains os influençaient des parties du corps qui ne leur correspondaient pas, ce qui provoquait des déformations incorrectes lors des mouvements. On a donc repris toute l'armature en veillant à ce que les os de la tête, des bras et des jambes n'agissent que sur les parties du corps correspondantes.

Une fois cette étape terminée, on a réalisé les différentes animations du personnage. On a commencé par l'animation lorsque le joueur est immobile. Celle-ci donne un léger effet d'écrasement au personnage afin de renforcer l'aspect cartoon, avec des bras qui se balancent légèrement pour donner l'impression qu'il attend. On s'est ensuite occupé de l'animation de course, dans laquelle on a repris le mouvement classique de la marche avec les bras opposés au mouvement des jambes afin de rendre l'animation plus naturelle.

Comme Ursina ne permet pas d'effectuer une interpolation automatique entre deux animations, on a également dû créer deux animations de transition afin d'assurer un passage fluide entre l'animation immobile et l'animation de déplacement.

On s'est également chargé de l'intégration des différentes animations dans le jeu avec la mise en place des transitions. Pour cela, on a utilisé les acteurs animés, qui permettent de gérer les animations des personnages. On a fait en sorte que lorsque le joueur se déplace, l'animation de marche se lance automatiquement, puis que l'animation d'inactivité se joue lorsqu'il s'arrête. Afin d'obtenir un rendu fluide, on utilise une temporisation permettant d'attendre la fin de l'animation de transition avant de lancer l'animation suivante.

Concernant le marchand, celui-ci possède également une animation jouée en boucle. Elle reprend presque le même principe que celle du joueur immobile, avec uniquement quelques mouvements supplémentaires au niveau de la tête afin de donner davantage de vie au personnage.

On s'est également occupé des shaders de l'eau, avec une animation des vagues réalisée à partir de différents cercles qui se propagent. L'ensemble est ensuite pixelisé afin de mieux correspondre au style graphique du jeu et d'obtenir un rendu cohérent avec l'esthétique générale.

## 5.7 Musiques

### 5.7.1 Bilan du Team Lead

Ayant déjà réalisé l'intégralité des compositions lors des précédentes soutenances, cette étape a principalement consisté à les intégrer au jeu. J'ai d'abord associé au menu principal le thème musical du jeu, qui se lance automatiquement au démarrage. J'ai ensuite implémenté une musique d'ambiance jouée en boucle durant les phases de jeu, hors menus.

Neuf autres musiques avaient également été composées pour des scènes et contextes particuliers : ambiance de casino, scène de fin, entre autres. Ces séquences n'ayant finalement pas été intégrées au projet, j'ai décidé de réutiliser ces morceaux en mettant en place un système de liste de lecture variant aléatoirement la musique de fond. Cette fonctionnalité a cependant entraîné plusieurs problèmes techniques : la correction de certains dysfonctionnements en faisait apparaître de nouveaux. Le système n'est donc pas encore totalement finalisé.

Concernant les effets sonores, j'ai ajouté un bruit de tir durant les combats ainsi que des chants d'oiseaux afin de rendre l'atmosphère plus vivante et immersive. J'ai récupéré ces sons sur une banque audio libre de droits (Pixabay) avant d'en retravailler certains sur FL Studio afin d'y ajouter différents effets sonores.

## 5.8 Rendus et packaging

### 5.8.1 Bilan du Team Lead

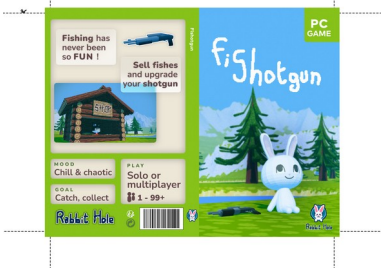
Concernant les rendus graphiques destinés à la communication du projet, j'ai retravaillé la scène du jeu sur *Blender* en y intégrant l'environnement complet, le personnage principal, les nouveaux arbres ainsi que le magasin. J'ai d'abord mis en scène le personnage principal en position assise avant de réaliser le rendu final de la scène.

J'ai ensuite retravaillé ce rendu sur *Adobe Photoshop* : correction des couleurs, ajout de flou, de grain et divers ajustements visuels. J'ai par la suite décliné cette scène, avec et sans logo, dans plusieurs formats : couverture, icône, bannière, entre autres.

Une fois cela terminé, j'ai conçu la jaquette du jeu en m'inspirant du modèle des jeux Nintendo Switch. J'ai ensuite imprimé le visuel et l'ai inséré dans une véritable boîte de jeu *Switch*. J'y ai également intégré la clé contenant le jeu ainsi qu'un manuel d'utilisation fourni par le Deputy Lead. Le projet dispose désormais d'une version matérielle complète, prête à être utilisée.






## 5.9 Physique

### 5.9.1 Bilan Noah

Après la deuxième soutenance, nous avons réalisé que la physique ne fonctionnait pas sur tous les ordinateurs. À la fin, elle ne fonctionnait plus que sur ma machine. Nous soupçonnons des différences d'environnement ou de versions, mais nous n'avons pas trouvé le problème exact. J'ai donc dû refaire la physique du jeu une nouvelle fois. Je me suis aidé du code que j'avais écrit avant, mais plusieurs nouveaux bugs sont apparus. Comme le projet avançait en parallèle, les merge conflicts étaient beaucoup trop compliqués à régler. J'ai donc dû repartir d'une branche à jour pour recoder une physique fonctionnelle.

Je me suis aussi occupé des BusSpots. Ce sont les panneaux que vous pouvez apercevoir à plusieurs endroits sur la carte. Ils permettent de se téléporter directement devant le shop.

Pour cela, j'ai recréé une classe BusSpot qui s'inspire de la classe FishingSpot. J'ai aussi dû créer différents paquets pour indiquer au serveur que la téléportation est normale et qu'il n'a pas besoin de corriger la position. J'ai ensuite placé les panneaux à différents endroits de la carte et je leur ai donné les coordonnées du shop comme destination de téléportation.

## 5.10 Organisation

Dans le tableau suivant, vous pourrez retrouver les différents avancements qui ont été faits, avec le pourcentage relatif des avancements et retards.

| Tâches | Soutenance 1 | Différence | Avancée actuelle |
|---|---|---|---|
| Assets | 100 % | **0** | Tous les assets ont été finis et réalisés dans les temps. |
| Pêche | 100 % | **0** | Le système de pêche a été complété. |
| Site Web | 100 % | **0** | La page d'accueil, l'architecture et le fonctionnement global du site web ont été réalisés. |

| Tâches | Soutenance 1 | Différence | Avancée actuelle |
|---|---|---|---|
| Mouvements | 70 % | **-30** | La physique du jeu n'était pas stable, nous avons donc recodé une dernière fois la physique de manière plus simple et stable et sans l'axe vertical car il est inutile dans notre jeu. |
| Interactions multijoueur (tchat) | 80 % | **-20** | Tout le tchat et les déplacements ont été codés, mais, faute de temps, il n'a pas été possible de faire les différentes émotes. |
| Networking | 100 % | **0** | Comme la dernière fois, le networking était déjà terminé à la dernière soutenance. |
| Interface | 90 % | **-10** | Toutes les interfaces ont été terminées sauf celle des paramètres. |
| IA des poissons | 100 % | **0** | Les poissons s'orientent fluidement vers leur prochaine position et réagissent dynamiquement. |
| Caméra | 100 % | **0** | La caméra était déjà fonctionnelle à 100 % lors de la dernière soutenance. |
| Carte (environnement) | 100 % | **0** | Le modèle 3D de la carte est fini et implémenté dans le jeu, les arbres et le magasin ont été placés avec les points de pêche. |
| Boutique | 95 % | **-5** | La boutique a été finie avec la possibilité de vendre les poissons et améliorer le fusil à pompe. |
| Marketing (boîtier, entreprise…) | 100 % | **0** | On peut télécharger le jeu depuis le site web, boîtier réalisé et imprimé. |

| Avance | Retard | Total |
|---|---|---|
| 0 | -65 | -65 |

Comme il est possible de le constater, beaucoup de retard s'est accumulé à cause du manque de temps des différents collaborateurs, ce qui fait que certains objectifs n'ont malheureusement pas pu être atteints.

# 6 Conclusion

Ce projet représente l'aboutissement d'un travail de longue durée ayant mobilisé à la fois des compétences techniques, créatives et organisationnelles. À travers Fishotgun, nous avons cherché à concevoir un jeu complet et cohérent, intégrant une boucle de gameplay basée sur la pêche, la progression du joueur et l'amélioration de son équipement, le tout dans un univers visuel et sonore travaillé.

Au-delà du simple développement d'un jeu, ce projet nous a permis de structurer une architecture complexe reposant sur un système multijoueur, une synchronisation serveur-client rigoureuse et une gestion centralisée des données afin de garantir la stabilité et l'intégrité des informations de jeu. Chaque fonctionnalité, qu'il s'agisse du Fishodex, de la boutique, des déplacements ou des interactions, a nécessité une réflexion approfondie pour s'intégrer correctement dans l'ensemble.

Les différentes étapes du développement ont également mis en évidence les contraintes réelles d'un projet de cette ampleur. La gestion du temps, les problèmes d'intégration entre les différentes branches de développement, ainsi que les limitations techniques du moteur utilisé ont parfois imposé des choix difficiles. Certaines fonctionnalités ont dû être simplifiées ou abandonnées afin de garantir la stabilité globale du jeu et de respecter les délais.

Malgré ces contraintes, l'équipe a su maintenir une cohérence globale dans la direction artistique et dans les mécaniques de jeu. L'univers créé repose sur une identité forte, à la fois originale et lisible, soutenue par un travail conséquent sur les assets 3D et 2D, les animations et les effets visuels. Cette cohérence renforce l'immersion du joueur et donne une véritable personnalité au projet.

Sur le plan technique, ce projet nous a permis de progresser sur des aspects essentiels du développement de jeux vidéo, notamment la gestion du réseau, l'optimisation des systèmes, la création d'interfaces interactives et la mise en place de mécaniques de gameplay fluides et réactives. Il a également renforcé notre capacité à travailler en équipe, à répartir les tâches de manière efficace et à communiquer sur des systèmes interconnectés.

En conclusion, même si l'ensemble des objectifs initiaux n'a pas pu être atteint à 100 %, le projet Fishotgun constitue une réalisation complète et fonctionnelle, reflétant un travail collectif important et une réelle montée en compétence de l'équipe. Il représente une expérience formatrice aussi bien sur le plan technique que sur le plan humain, et constitue une base solide pour de futurs projets de développement plus ambitieux.